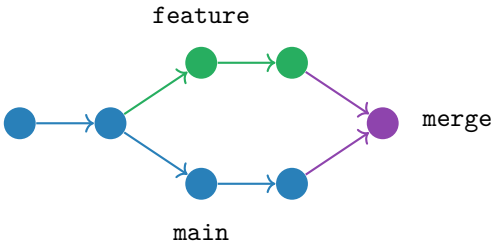
GitOus

A Beginner's Guide to
Learning Git Through a Python Project

Master Version Control by Building
TaskFlow: A CLI Task Manager



Where Python is the Vehicle,
and Git is the Destination

Made by Oussama BOUSSAHLA

March 2026

Prerequisites

Before You Begin

This guide assumes you have:

- **Python 3.8+** installed on your system
- **Basic Python knowledge**: functions, classes, file I/O, JSON handling
- **Command line familiarity**: navigating directories, running commands
- **Git installed**: verify with `git -version`
- A **text editor** or IDE (VS Code, PyCharm, etc.)

What You'll Build

TaskFlow – A command-line task management application featuring:

- Create, list, update, and delete tasks
- Priority levels and due dates
- Search and filter functionality
- Export/import capabilities
- A clean, modular Python codebase

What You'll Learn

- Git's three-tree architecture and internal mechanics
- Branching strategies and workflow patterns
- Merge conflicts: how to create and resolve them
- Rebase vs. merge decisions
- Recovery techniques using reflog
- GitHub and GitLab collaboration workflows
- CI/CD basics with GitHub Actions and GitLab CI

Contents

1 Introduction

1.1 Why GitOus Tutorial Exists

Most Git tutorials teach commands in isolation. You memorize `git merge`, `git rebase`, `git reset`, but when chaos strikes in a real project, you freeze. You don't know *why* things work the way they do, just that sometimes they work and sometimes they don't.

This tutorial is different.

You will build **TaskFlow**, a real, working CLI task management application in Python. Every feature you add will teach you a Git concept by forcing you to *feel* what happens to the commit graph. You'll make mistakes on purpose, break things intentionally, and learn how to fix them.

By the end, you won't just know Git commands. **You'll predict what pointers move before you run them.**

Why Version Control Matters

Imagine you're working on a Python project and you make a change that breaks everything. Without version control:

- You frantically try to remember what you changed
- You might lose hours of work trying to fix it
- You have no way to safely experiment with new features

With Git:

- Every change is recorded with a timestamp and description
- You can instantly revert to any previous state
- Multiple people can work on the same codebase without overwriting each other
- You can experiment freely knowing you can always go back

Companies like Google, Microsoft, and every tech startup rely on Git daily. Learning it properly will make you a more confident and employable developer.

1.2 The TaskFlow Project

What You'll Build

TaskFlow is a command-line task manager that demonstrates professional Python practices while teaching Git concepts:

- **Core Features**: Create, read, update, delete tasks (CRUD operations)
- **Advanced Features**: Priorities, due dates, tags, search, export
- **Code Structure**: Modular design with separate files for logic, storage, and CLI
- **Data Persistence**: JSON-based storage with proper error handling

By the end of this tutorial, you'll have a fully functional application *and* a deep understanding of Git.

1.3 How to Use This Tutorial

This tutorial is designed for active learning. Don't just read –> **do**.

1. **Read** the feature description and Git concept
2. **Predict** the commit graph (before running commands)
3. **Execute** the Git commands on your computer
4. **Visualize** the result (diagrams provided for comparison)
5. **Understand** what moved and why

⚠ Never Skip the Prediction Step

The prediction step is where learning happens. If you just copy commands without thinking, you'll memorize but not understand. Before every Git command, pause and ask yourself: "What do I expect to happen?"
This single habit separates developers who *use* Git from developers who *understand* Git.

1.4 Git's Mental Model: The Foundation

Before writing any code, let's build the mental model that will make everything else click. Git has a simple but powerful architecture built on three concepts.

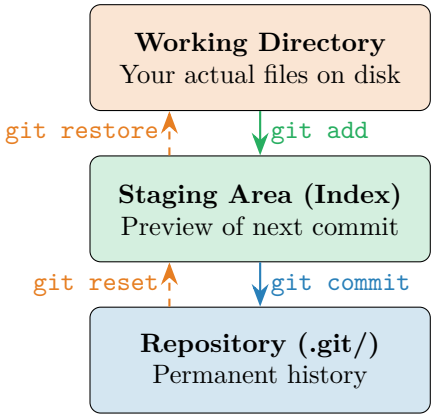


Figure 1.1: Git's Three-Tree Architecture – the foundation of everything

The Three Trees Explained

Think of Git as having three separate "areas" where your code can exist:

1. **Working Directory**: This is what you see in your file explorer. When you open `taskflow.py` in VS Code, you're editing the working directory. These are your actual files.
2. **Staging Area** (also called the Index): Think of this as a "shopping cart" for your next commit. When you run `git add`, you're putting items in the cart. Nothing is permanent yet – you can add more items or remove them.
3. **Repository**: This is Git's permanent database stored in the hidden `.git/` folder. When you run `git commit`, you're "checking out" your shopping cart – creating an immutable snapshot that becomes part of your project's history forever.

Why Three Areas?

You might wonder: why not just save directly from your files to the repository?
The staging area gives you **control**. Imagine you fixed a bug *and* started a new feature in the same coding session. With the staging area, you can:

- Stage only the bug fix files
- Commit them with message "fix: resolve login timeout"
- Then stage the feature files
- Commit them with message "feat: add user preferences"

This creates clean, logical commits instead of one messy "fixed bug and also started preferences" commit.

2 The Three-Tree Architecture

Now let's put theory into practice. We'll create our project and observe how files move between Git's three areas.

2.1 Understanding the Staging Area

The staging area is Git's most misunderstood feature. Most version control systems (like old-school SVN) go directly from working directory to repository. Git adds an intermediate step – and this is a *feature*, not a bug.

Why the Staging Area Exists

The staging area enables:

- **Selective commits**: Commit only some changes while keeping others for later
- **Review before commit**: Use `git diff -staged` to check exactly what you're about to commit
- **Atomic commits**: Build logical, focused commits piece by piece
- **Partial staging**: Stage parts of files using `git add -p`, not just entire files

2.2 Step 1: Initialize the TaskFlow Project

Open your terminal and create the project directory:

```
mkdir taskflow
cd taskflow
git init
```

The `git init` command creates a hidden `.git/` folder. This folder *is* your repository – it contains the entire history of your project.

Check the status to see the current state:

```
git status
```

```
On branch main

No commits yet

nothing to commit (create/copy files and track with "git add")
```

Current State of the Three Trees

Right now, all three areas are empty:

- **Working Directory**: Empty (no files yet)
- **Staging Area**: Empty (nothing staged)
- **Repository**: Empty (no commits yet)

Git is watching this folder, but there's nothing to track.

2.3 Step 2: Create the First Python Files

Let's create the main entry point. Create a file called `taskflow.py`:

Listing 2.1: taskflow.py – Initial structure (v0.1.0)

```python
#!/usr/bin/env python3
"""TaskFlow - A CLI Task Management Application

This is the main entry point for TaskFlow. It handles command-line
arguments and routes them to the appropriate functions.

Author: Oussama BOUSSAHLA
Version: 0.1.0
"""

__version__ = "0.1.0"
__author__ = "Oussama BOUSSAHLA"


def show_help():
    """Display detailed help information.

    Shows all available commands with descriptions and examples.

    Returns:
        None
    """
    print(f"TaskFlow v{__version__}")
    print("Your command-line task manager")
    print("\nUsage: python taskflow.py <command> [options]")
    print("\nCommands:")
    print("  add       Add a new task")
    print("  list      List all tasks")
    print("  help      Show this help message")


def main():
    """Main entry point for TaskFlow.

    This function displays the welcome message and usage instructions.
    We'll expand this to handle actual commands in later steps.

    Returns:
        None
    """
    show_help()


if __name__ == "__main__":
    main()
```

Now create `storage.py` to handle data persistence:

Listing 2.2: storage.py – Data persistence module (v0.1.0)

```
"""Storage module for TaskFlow - handles JSON data persistence.

This module provides functions to load and save tasks to a JSON file.
Using JSON makes our data human-readable and easy to debug.

Author: Oussama BOUSSAHLA
Version: 0.1.0
"""

import json
from pathlib import Path

# Path to our data file
TASKS_FILE = Path("tasks.json")


def load_tasks():
    """Load tasks from the JSON file.

    Returns:
        dict: Contains 'tasks' list and 'next_id' counter.
              Returns default structure if file doesn't exist.
    """
    if TASKS_FILE.exists():
        try:
            with open(TASKS_FILE, 'r', encoding='utf-8') as f:
                return json.load(f)
        except json.JSONDecodeError:
            print("Warning: Could not parse tasks.json. Creating new
                file.")
            return {"tasks": [], "next_id": 1}
    return {"tasks": [], "next_id": 1}


def save_tasks(data):
    """Save tasks to the JSON file.

    Args:
        data: Dictionary with 'tasks' list and 'next_id' counter.
    """
    with open(TASKS_FILE, 'w', encoding='utf-8') as f:
        json.dump(data, f, indent=2, ensure_ascii=False)
```

2.4 Step 3: Understanding Tracked vs Untracked

Now let's see what Git thinks about our new files:

```
git status
```

```
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        storage.py
        taskflow.py

nothing added to commit but untracked files present
```

Tracked vs Untracked Files

Git categorizes files into two groups:

- **Untracked**: Git sees the file exists but isn't managing it. It's like a book sitting on your desk that hasn't been added to your library catalog yet.
- **Tracked**: Git is actively monitoring this file for changes. Any modifications will be detected and reported by `git status`.

Files become tracked when you first `git add` them. Once tracked, Git will notice every change you make.

2.5 Step 4: Stage One File

Let's stage only `taskflow.py` to see selective staging in action:

```
git add taskflow.py
git status
```

```
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   taskflow.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        storage.py
```

Notice how Git now shows two separate sections:

- **Changes to be committed**: `taskflow.py` is in our staging area, ready to go
- **Untracked files**: `storage.py` is still not being tracked

Prediction Challenge

If you run `git commit` right now, what will be in the commit?
Answer: Only `taskflow.py`. The staging area determines what goes into a commit, not the working directory. This is the power of selective commits!

2.6 Step 5: Understanding git diff

Git provides two diff commands that compare different areas:

Figure 2.1: The Two Diff Commands compare different areas

```
# Show unstaged changes (Working Directory vs Staging Area)
git diff

# Show staged changes (Staging Area vs last commit)
git diff --staged
```

When to Use Each Diff

- Use `git diff` to see what you've changed but haven't staged yet
- Use `git diff -staged` to review exactly what will go into your next commit

Always run `git diff -staged` before committing to avoid surprises!

2.7 Step 6: The Same File in Two States

Here's something that confuses many beginners. Let's add a function to our already-staged file:

Add this function to `taskflow.py` before the `main()` function:

```
def show_help():
    """Display detailed help information.

    Shows all available commands with examples.
    """
    print("TaskFlow - CLI Task Manager")
    print("\nAvailable commands:")
    print("  add <title>     Add a new task")
    print("  list            List all tasks")
    print("  done <id>       Mark task as complete")
    print("  delete <id>     Delete a task")
```

Now check the status:

```
git status
```

```
On branch main

No commits yet

Changes to be committed:
        new file:   taskflow.py

Changes not staged for commit:
        modified:   taskflow.py

Untracked files:
        storage.py
```

⚠ The Same File in Two States!

Look carefully: `taskflow.py` appears in *both* "staged" and "unstaged"! This isn't an error – it's showing you exactly how Git works:

- The **staging area** has the *old* version (without `show_help`)
- The **working directory** has the *new* version (with `show_help`)

If you commit now, only the old version (without the new function) will be committed. You must `git add` again to update the staging area with your latest changes.

2.8 Step 7: The First Commit

Let's stage all our files and create our first commit:

```
git add taskflow.py storage.py
git commit -m "Initial commit: TaskFlow project structure"
```

```
[main (root-commit) a1b2c3d] Initial commit: TaskFlow project
   structure
 2 files changed, 52 insertions(+)
 create mode 100644 storage.py
 create mode 100644 taskflow.py
```

Notice the `(root-commit)` label – this is Git's first commit. Also notice there's no `HEAD` reference shown yet; we'll learn about HEAD at the end of this chapter.

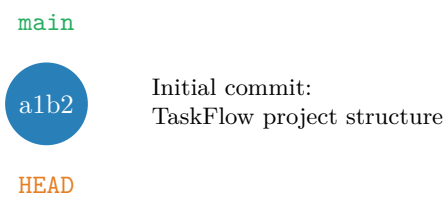


Figure 2.2: Your First Commit – the beginning of history

What Just Happened?

When you ran `git commit`:

1. Git took a snapshot of everything in the staging area
2. Created a unique hash (`a1b2c3d`) to identify this snapshot
3. Created a "commit object" storing: the snapshot, your name, email, timestamp, and message
4. Moved the `main` branch pointer to this new commit
5. Cleared the staging area (it now matches the repository)

This commit is now **immutable** – it will exist forever in your repository's history.

2.9 Step 8: Interactive Staging with git add -p

Now let's add a more substantial feature. Update `taskflow.py` with task creation and listing:

==code on the following page==

Listing 2.3: taskflow.py with task creation (v0.2.0)

```python
#!/usr/bin/env python3
"""TaskFlow - A CLI Task Management Application

Author: Oussama BOUSSAHLA
Version: 0.2.0
"""

import sys
from datetime import datetime
from storage import load_tasks, save_tasks

__version__ = "0.2.0"
__author__ = "Oussama BOUSSAHLA"


def add_task(title, priority="medium"):
    """Add a new task to the task list.

    Args:
        title (str): The task description
        priority (str): Priority level - 'low', 'medium', or 'high'

    Returns:
        None
    """
    if not title or not title.strip():
        print("Error: Task title cannot be empty.")
        return

    if priority not in ('low', 'medium', 'high'):
        print(f"Error: Invalid priority '{priority}'. Using 'medium'.
            ")
        priority = 'medium'

    data = load_tasks()
    task = {
        "id": data["next_id"],
        "title": title.strip(),
        "priority": priority,
        "completed": False,
        "created": datetime.now().isoformat()
    }
    data["tasks"].append(task)
    data["next_id"] += 1
    save_tasks(data)
    print(f"Task #{task['id']} added: {title}")


def list_tasks():
    """Display all tasks with their status.

    Returns:
        None
    """
    data = load_tasks()
    if not data["tasks"]:
        print("\nNo tasks yet. Add one with: python taskflow.py add <
```

```
            title>")
        return

    print("\nYour Tasks:")
    print("-" * 40)
    for task in data["tasks"]:
        status = "[x]" if task["completed"] else "[ ]"
        print(f"{status} #{task['id']} {task['title']} ({task['
            priority']})")


def show_help():
    """Display help information.

    Returns:
        None
    """
    print(f"TaskFlow v{__version__}")
    print("Usage: python taskflow.py <command> [args]")
    print("\nCommands:")
    print("  add <title>     Add a new task")
    print("  list            List all tasks")
    print("  help            Show this help")


def main():
    """Main entry point - routes commands to functions.

    Returns:
        None
    """
    if len(sys.argv) < 2:
        show_help()
        return

    command = sys.argv[1].lower()

    if command == "add" and len(sys.argv) >= 3:
        title = " ".join(sys.argv[2:])
        add_task(title)
    elif command == "list":
        list_tasks()
    elif command == "help":
        show_help()
    else:
        print(f"Unknown command: {command}")
        print("Run 'python taskflow.py help' for usage.")


if __name__ == "__main__":
    main()
```

Now try interactive staging to review each change:

```
git add -p taskflow.py
```

Git will show you each "hunk" (section) of changes and ask what to do:

```
@@ -1,15 +1,45 @@
+import sys
+from storage import load_tasks, save_tasks
...
Stage this hunk [y,n,q,a,d,s,e,?]?
```

Interactive Staging Options

- **y** Yes, stage this hunk
- **n** No, don't stage this hunk
- **s** Split into smaller hunks (if possible)
- **e** Manually edit the hunk
- **q** Quit (don't stage any remaining hunks)
- **a** Stage this hunk and all remaining hunks
- **?** Show help for all options

Interactive staging is incredibly useful when you've made multiple unrelated changes and want to create separate, logical commits.

2.10 Step 9: Unstaging with git reset

Made a mistake? You can unstage files without losing your work:

```
# Unstage a specific file
git reset taskflow.py

# Unstage everything
git reset
```

This copies the file from the last commit back to the staging area, effectively "unstaging" it. **Your working directory is untouched** – your changes are still there.

Reset vs Restore

Git provides multiple ways to undo:

- `git reset <file>`: Unstages a file (staging → matches repo)
- `git restore <file>`: Discards working directory changes (working → matches staging)
- `git restore -staged <file>`: Same as reset (newer syntax)

2.11 Step 10: Commit the Feature

```
git add taskflow.py
git commit -m "feat: Add task creation and listing functionality"
```

Figure 2.3: Linear History with Two Commits

2.12 The .gitignore File – Keeping Your Repository Clean

Not every file in your project directory should be tracked by Git. Some files are generated automatically, contain sensitive information, or are specific to your local environment. The `.gitignore` file tells Git which files to ignore.

Why Use .gitignore?

The Purpose of .gitignore

The `.gitignore` file prevents certain files from being tracked by Git:

- **Generated files**: Compiled code, build artifacts, PDFs
- **Dependencies**: `node_modules/`, `venv/`, Python packages
- **OS files**: `.DS_Store` (macOS), `Thumbs.db` (Windows)
- **IDE files**: `.vscode/`, `.idea/`, editor configurations
- **Sensitive data**: API keys, passwords, configuration files
- **Personal files**: `tasks.json`, local notes, temporary files

Golden rule: Only commit source code and documentation, not generated files.

Creating a .gitignore File

Create a file named `.gitignore` in your project root:

```
# Create .gitignore file
touch .gitignore

# Or on Windows
type nul > .gitignore
```

.gitignore for TaskFlow

Here's a comprehensive `.gitignore` for our TaskFlow project:

Listing 2.4: .gitignore for TaskFlow

```
# TaskFlow specific files
tasks.json
tasks_backup.json
tasks_export.txt
*.txt

# Python cache and bytecode
__pycache__/
*.py[cod]
*$py.class
*.so
.Python

# Virtual environments
venv/
env/
ENV/
.venv
virtualenv/

# IDE and editor files
.vscode/
.idea/
*.swp
*.swo
*~
.project
.pydevproject

# Operating system files
.DS_Store
.DS_Store?
._*
.Spotlight-V100
.Trashes
ehthumbs.db
Thumbs.db
Desktop.ini

# Testing and coverage
.pytest_cache/
.coverage
htmlcov/
.tox/
.hypothesis/

# Distribution and packaging
dist/
build/
*.egg-info/
*.egg

# Documentation builds
docs/_build/
site/

# Logs and databases
*.log
*.sqlite
*.db
```

.gitignore Pattern Rules

Pattern	What It Matches
`*.txt`	All files ending in `.txt`
`build/`	The `build` directory and everything in it
`!important.txt`	Exception: DO track this file
`*.py[cod]`	Files ending in `.pyc`, `.pyo`, or `.pyd`
`temp*`	Files starting with `temp`
`/TODO`	Only `TODO` in root, not `subdir/TODO`
`doc/*.txt`	`.txt` files in `doc/`, not `doc/server/`
`doc/**/*.pdf`	All `.pdf` files in `doc/` and subdirectories
`#`	Comment line (ignored)

Table 2.1: Common .gitignore Patterns

Adding .gitignore to Your Project

```
# Create the .gitignore file
echo "tasks.json" > .gitignore
echo "*.txt" >> .gitignore
echo "__pycache__/" >> .gitignore

# Add it to Git
git add .gitignore
git commit -m "chore: Add .gitignore file"
```

⚠ Files Already Tracked

If you already committed a file before adding it to `.gitignore`, Git will continue tracking it. You need to explicitly remove it:

```
# Remove from Git but keep locally
git rm --cached tasks.json

# Remove entire directory from Git
git rm -r --cached __pycache__/

# Commit the removal
git commit -m "chore: Remove tracked files that should be
    ignored"
```

The `-cached` flag removes files from Git's index but keeps them on your disk.

Global .gitignore

You can create a global `.gitignore` for files you *never* want to commit:

```
# Create global gitignore
git config --global core.excludesfile ~/.gitignore_global

# Add OS and editor files
echo ".DS_Store" >> ~/.gitignore_global
```

```
echo ".vscode/" >> ~/.gitignore_global
echo "*.swp" >> ~/.gitignore_global
```

Use gitignore.io

Visit https://gitignore.io to generate `.gitignore` files for your technology stack. For example, search for "Python, VSCode, macOS" and get a comprehensive template with hundreds of patterns already included.

Common Mistakes with .gitignore

1. **Adding .gitignore after committing files**
   - Fix: Use `git rm -cached` to untrack them
2. **Forgetting the leading dot**
   - Wrong: `gitignore`
   - Right: `.gitignore`
3. **Ignoring too much**
   - Don't ignore source code!
   - Only ignore generated/temporary files
4. **Committing sensitive data**
   - ALWAYS add secrets to `.gitignore` before committing
   - If you commit a secret, it stays in history even after removal

Checking What's Ignored

```
# See which files are ignored
git status --ignored

# Check if a specific file is ignored
git check-ignore -v tasks.json

# List all ignored files in the repository
git ls-files --others --ignored --exclude-standard
```

Real-World .gitignore Strategy

In professional projects:

- **Start early**: Create `.gitignore` in your first commit
- **Use templates**: Start with gitignore.io or GitHub templates
- **Team coordination**: Agree on what to ignore (commit the `.gitignore`)
- **Document exceptions**: Comment why you're tracking unusual files
- **Review regularly**: Update as your project evolves

Example first commit:

```
git init
git add .gitignore README.md
```

```
git commit -m "Initial commit: Project structure"
```

Exercise: TaskFlow .gitignore

Practice with .gitignore

For the TaskFlow project:

1. Create a `.gitignore` file
2. Add patterns to ignore `tasks.json`, Python cache, and OS files
3. If you already committed `tasks.json`, remove it from Git
4. Verify it's ignored: `git status -ignored`
5. Commit your `.gitignore`

Verify success:

- `tasks.json` should appear in `git status -ignored`
- `.gitignore` itself should be tracked and committed

2.13 Where You Are: A First Look at HEAD

Before we dive into branches, you need to understand one more critical concept: **where you currently are** in your repository.

HEAD: Your Current Location

HEAD is Git's way of tracking your current position in the project's history. Think of it as:

- A **bookmark** showing which page you're on
- A **GPS marker** showing "You are here" on a map
- Your **cursor position** in a document

When you make a commit, Git adds it to wherever HEAD is pointing.

Check where HEAD points:

```
git log --oneline
```

```
f6g7h8i (HEAD -> main) feat: Add task creation and listing
a1b2c3d Initial commit: TaskFlow project structure
```

The `HEAD -> main` notation tells you:

- HEAD is currently pointing to the `main` branch
- The `main` branch points to commit `f6g7h8i`
- Any new commits will be added here

Remember This

For now, just remember: **HEAD = where you are**. In the next chapter, we'll see how HEAD moves when you switch between branches and how it determines where your commits go.

2.14 Key Commands Summary

Command	What It Does
`git add <file>`	Stage a file (copy from working directory to staging area)
`git add -p`	Interactively stage parts of files
`git add -A`	Stage all changes (new, modified, and deleted files)
`git reset <file>`	Unstage a file (keep working directory changes)
`git diff`	Show unstaged changes (working vs staging)
`git diff -staged`	Show staged changes (staging vs last commit)
`git status`	Show the state of all three trees
`git commit -m "msg"`	Create a commit from the staging area

Table 2.2: Essential Staging Area Commands

3 Branches as Lightweight Pointers

Branches are Git's killer feature. They let you work on multiple things simultaneously without them interfering with each other. But to use them effectively, you need to understand what they really are.

As we learned in Chapter 2, HEAD tracks your current position. When you create a branch, you're creating a new pointer that HEAD can point to. Let's see this in action.

3.1 Concept: Branches Are Not Containers

Here's the mental shift that makes Git click: **a branch is just a pointer**.

What is a Branch, Really?

A branch is **nothing more than a pointer to a commit**. That's it.
When you create a branch, Git literally just writes a 41-character file:
`.git/refs/heads/feature-list`
`Contents: d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u1v2w3`
No files are copied. No history is duplicated. Creating a branch takes milliseconds because Git is just creating a tiny text file containing a commit hash.
This is why Git branches are called "lightweight" – they're just pointers, not copies.

Branches in the Real World

In professional development, branches enable parallel workflows:

- **main/master**: The stable, production-ready code
- **develop**: Integration branch for features
- **feature/xyz**: Individual features in development
- **hotfix/urgent**: Emergency fixes for production
- **release/v1.2**: Preparing a specific version for release

A typical day might involve: fixing a bug on `hotfix`, switching to `feature/login` to continue work, then reviewing a colleague's code on `feature/payments`.

3.2 Step 11: Create a Feature Branch

Let's add a "complete task" feature on a separate branch:

```
# Create and switch to new branch in one command
git checkout -b feature/complete-task

# Or using modern Git (2.23+):
# git switch -c feature/complete-task
```

What Just Happened?

This single command did two things:

1. **Created** a new branch pointer called `feature/complete-task`
2. **Moved HEAD** to point to this new branch

Remember from Chapter 2: HEAD shows your current location. When you create and switch to a new branch, HEAD moves to point to that branch.
You can verify this:

```
# See which branch you're on (where HEAD points)
git branch
# The * marks your current branch

# See what HEAD points to
cat .git/HEAD
# Shows: ref: refs/heads/feature/complete-task
```

Prediction Challenge

Before checking `git log`, predict:

- Where does `feature/complete-task` point?
- Where does `main` point?
- Where does `HEAD` point?

Think about it before scrolling down!

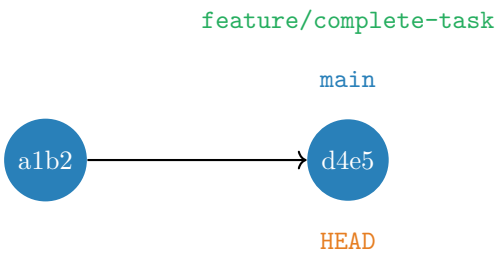


Figure 3.1: Both Branches Point to the Same Commit

Answer: All three point to the same commit! Creating a branch doesn't create a new commit – it just creates a new pointer to an existing commit.

3.3 Step 12: Implement the Feature

Add the complete task functionality to `taskflow.py`:

==code on the following page==

Listing 3.1: taskflow.py with complete task (v0.3.0)

```
#!/usr/bin/env python3
"""TaskFlow - A CLI Task Management Application

Author: Oussama BOUSSAHLA
Version: 0.3.0
"""

import sys
from datetime import datetime
from storage import load_tasks, save_tasks

__version__ = "0.3.0"
__author__ = "Oussama BOUSSAHLA"


def add_task(title, priority="medium"):
    """Add a new task to the task list.

    Args:
        title (str): The task description
        priority (str): Priority level - 'low', 'medium', or 'high'

    Returns:
        None
    """
    if not title or not title.strip():
        print("Error: Task title cannot be empty.")
        return

    if priority not in ('low', 'medium', 'high'):
        print(f"Error: Invalid priority '{priority}'. Using 'medium'.
            ")
        priority = 'medium'

    data = load_tasks()
    task = {
        "id": data["next_id"],
        "title": title.strip(),
        "priority": priority,
        "completed": False,
        "created": datetime.now().isoformat()
    }
    data["tasks"].append(task)
    data["next_id"] += 1
    save_tasks(data)
    print(f"Task #{task['id']} added: {title}")


def list_tasks():
    """Display all tasks with their status.

    Returns:
        None
    """
    data = load_tasks()
    if not data["tasks"]:
        print("\nNo tasks yet. Add one with: python taskflow.py add <
```

```
            title>")
        return

    print("\nYour Tasks:")
    print("-" * 40)
    for task in data["tasks"]:
        status = "[x]" if task["completed"] else "[ ]"
        print(f"{status} #{task['id']} {task['title']} ({task['
            priority']})")


def complete_task(task_id):
    """Mark a task as completed.

    Args:
        task_id (int): The ID of the task to complete

    Returns:
        None
    """
    data = load_tasks()

    for task in data["tasks"]:
        if task["id"] == task_id:
            if task["completed"]:
                print(f"Task #{task_id} is already complete.")
                return

            task["completed"] = True
            task["completed_at"] = datetime.now().isoformat()
            save_tasks(data)
            print(f"Task #{task_id} marked as complete!")
            return

    print(f"Error: Task #{task_id} not found.")


def show_help():
    """Display help information.

    Returns:
        None
    """
    print(f"TaskFlow v{__version__}")
    print("Usage: python taskflow.py <command> [args]")
    print("\nCommands:")
    print("  add <title>     Add a new task")
    print("  list            List all tasks")
    print("  done <id>       Mark task as complete")
    print("  help            Show this help")


def main():
    """Main entry point - routes commands to functions.

    Returns:
        None
    """
```

```
    if len(sys.argv) < 2:
        show_help()
        return

    command = sys.argv[1].lower()

    if command == "add" and len(sys.argv) >= 3:
        title = " ".join(sys.argv[2:])
        add_task(title)
    elif command == "list":
        list_tasks()
    elif command == "done" and len(sys.argv) >= 3:
        try:
            task_id = int(sys.argv[2])
            complete_task(task_id)
        except ValueError:
            print("Error: Task ID must be a number.")
    elif command == "help":
        show_help()
    else:
        print(f"Unknown command: {command}")
        print("Run 'python taskflow.py help' for usage.")


if __name__ == "__main__":
    main()
```

Commit the feature:

```
git add taskflow.py
git commit -m "feat: Add ability to mark tasks as complete"
```

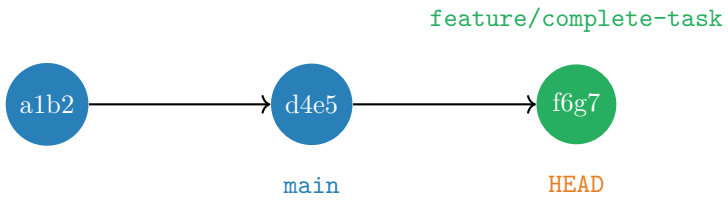


Figure 3.2: Feature Branch Has Advanced

Critical Observation: How HEAD Controls Commits

Notice what happened:

- `feature/complete-task` moved to the new commit (f6g7)
- `main` stayed at the old commit (d4e5)
- `HEAD` points to `feature/complete-task`

Key insight: When you commit, **only the branch that HEAD points to moves**. Other branches are unaffected.

This is how Git knows where to add your commit:

1. You run `git commit`
2. Git checks: "Where is HEAD pointing?" → `feature/complete-task`
3. Git creates a new commit as a child of the current commit
4. Git moves `feature/complete-task` forward to the new commit
5. HEAD follows along (since it points to `feature/complete-task`)

We'll explore what happens when HEAD *isn't* pointing to a branch in Chapter 4.

3.4 Step 13: Parallel Development

This is where branches shine. Let's switch back to main and start a different feature:

```
git checkout main
git checkout -b feature/delete-task
```

What Happened to HEAD?

When you ran `git checkout main`:

1. HEAD moved from `feature/complete-task` to `main`
2. Your working directory changed to match the `main` commit
3. The complete task code "disappeared" (it's safe in the feature branch)

Then `git checkout -b feature/delete-task`:

1. Created a new branch pointing at the same commit as `main`
2. Moved HEAD to point to this new branch

HEAD is now on `feature/delete-task`, ready for you to make commits.

Your working directory now reflects the state at the `main` commit – the complete task code is "gone" (it's safely stored in the feature branch).

Add delete functionality:

==code on the following page==

Listing 3.2: taskflow.py with delete task (v0.4.0)

```
#!/usr/bin/env python3
"""TaskFlow - A CLI Task Management Application

Author: Oussama BOUSSAHLA
Version: 0.4.0
"""

import sys
from datetime import datetime
from storage import load_tasks, save_tasks

__version__ = "0.4.0"
__author__ = "Oussama BOUSSAHLA"


def add_task(title, priority="medium"):
    """Add a new task to the task list.

    Args:
        title (str): The task description
        priority (str): Priority level - 'low', 'medium', or 'high'

    Returns:
        None
    """
    if not title or not title.strip():
        print("Error: Task title cannot be empty.")
        return

    if priority not in ('low', 'medium', 'high'):
        print(f"Error: Invalid priority '{priority}'. Using 'medium'.
            ")
        priority = 'medium'

    data = load_tasks()
    task = {
        "id": data["next_id"],
        "title": title.strip(),
        "priority": priority,
        "completed": False,
        "created": datetime.now().isoformat()
    }
    data["tasks"].append(task)
    data["next_id"] += 1
    save_tasks(data)
    print(f"Task #{task['id']} added: {title}")


def list_tasks():
    """Display all tasks with their status.

    Returns:
        None
    """
    data = load_tasks()
    if not data["tasks"]:
        print("\nNo tasks yet. Add one with: python taskflow.py add <
```

```
            title>")
57          return
58
59      print("\nYour Tasks:")
60      print("-" * 40)
61      for task in data["tasks"]:
62          status = "[x]" if task["completed"] else "[ ]"
63          print(f"{status} #{task['id']} {task['title']} ({task['
                priority']})")
64
65
66  def complete_task(task_id):
67      """Mark a task as completed.
68
69      Args:
70          task_id (int): The ID of the task to complete
71
72      Returns:
73          None
74      """
75      data = load_tasks()
76
77      for task in data["tasks"]:
78          if task["id"] == task_id:
79              if task["completed"]:
80                  print(f"Task #{task_id} is already complete.")
81                  return
82
83              task["completed"] = True
84              task["completed_at"] = datetime.now().isoformat()
85              save_tasks(data)
86              print(f"Task #{task_id} marked as complete!")
87              return
88
89      print(f"Error: Task #{task_id} not found.")
90
91
92  def delete_task(task_id):
93      """Delete a task by its ID.
94
95      Args:
96          task_id (int): The ID of the task to delete
97
98      Returns:
99          None
100     """
101     data = load_tasks()
102     original_count = len(data["tasks"])
103
104     # Filter out the task with the given ID
105     data["tasks"] = [t for t in data["tasks"] if t["id"] != task_id]
106
107     if len(data["tasks"]) < original_count:
108         save_tasks(data)
109         print(f"Task #{task_id} deleted.")
110     else:
111         print(f"Error: Task #{task_id} not found.")
112
```

```
113
114 def show_help():
115     """Display help information.
116
117     Returns:
118         None
119     """
120     print(f"TaskFlow v{__version__}")
121     print("Usage: python taskflow.py <command> [args]")
122     print("\nCommands:")
123     print("  add <title>     Add a new task")
124     print("  list            List all tasks")
125     print("  done <id>       Mark task as complete")
126     print("  delete <id>     Delete a task")
127     print("  help            Show this help")
128
129
130 def main():
131     """Main entry point - routes commands to functions.
132
133     Returns:
134         None
135     """
136     if len(sys.argv) < 2:
137         show_help()
138         return
139
140     command = sys.argv[1].lower()
141
142     if command == "add" and len(sys.argv) >= 3:
143         title = " ".join(sys.argv[2:])
144         add_task(title)
145     elif command == "list":
146         list_tasks()
147     elif command == "done" and len(sys.argv) >= 3:
148         try:
149             task_id = int(sys.argv[2])
150             complete_task(task_id)
151         except ValueError:
152             print("Error: Task ID must be a number.")
153     elif command == "delete" and len(sys.argv) >= 3:
154         try:
155             task_id = int(sys.argv[2])
156             delete_task(task_id)
157         except ValueError:
158             print("Error: Task ID must be a number.")
159     elif command == "help":
160         show_help()
161     else:
162         print(f"Unknown command: {command}")
163         print("Run 'python taskflow.py help' for usage.")
164
165
166 if __name__ == "__main__":
167     main()
```

```
git add taskflow.py
git commit -m "feat: Add task deletion functionality"
```

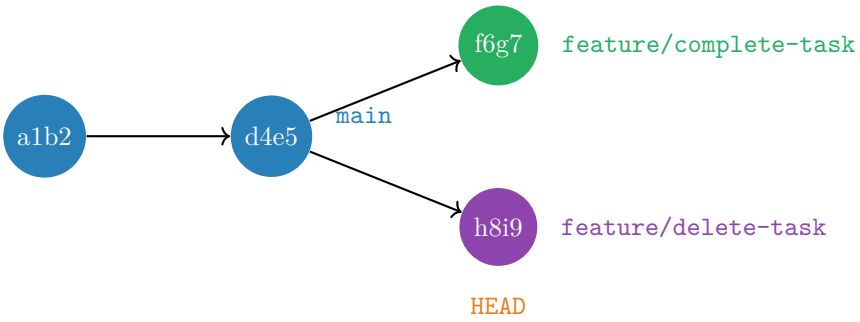


Figure 3.3: Diverging Branches – parallel development in action

This is the power of Git: two features being developed simultaneously, completely isolated from each other, sharing a common history.

4 Advanced HEAD: Detached State and Recovery

In Chapter 2, we introduced HEAD as your current location marker. In Chapter 3, we saw how HEAD moves when you switch branches and determines where your commits go. Now we'll explore HEAD's special states and how to use it for recovery when things go wrong.

4.1 Review: How HEAD Works Normally

Let's quickly review what we know about HEAD:

- **HEAD = your current position** in the Git timeline
- Normally, HEAD points to a branch name (e.g., HEAD → main)
- That branch points to a commit (e.g., main → a1b2c3d)
- When you commit, the branch moves forward and HEAD follows
- When you switch branches (`git checkout`), HEAD moves to the new branch

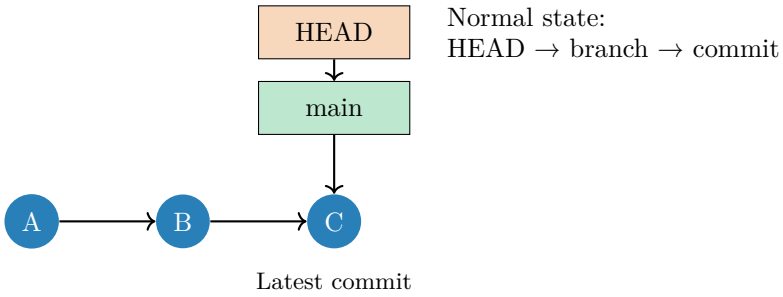


Figure 4.1: Normal HEAD State: HEAD → main → commit C

This is the normal, safe state where you should spend 99% of your time.

4.2 Detached HEAD: The Special State

What is Detached HEAD?

Detached HEAD is a special state where HEAD points **directly to a commit** instead of to a branch.

Normal state:

- HEAD → branch → commit
- Example: HEAD → main → a1b2c3d
- When you commit, the branch moves forward
- Commits are attached to a branch (safe)

Detached state:

- HEAD → commit (no branch in between)
- Example: HEAD → a1b2c3d directly
- When you commit, **no branch moves**
- New commits become "orphaned" (dangerous!)

Why does this state exist?

- To explore old versions of your code
- To run tests on a specific commit
- To make temporary experiments

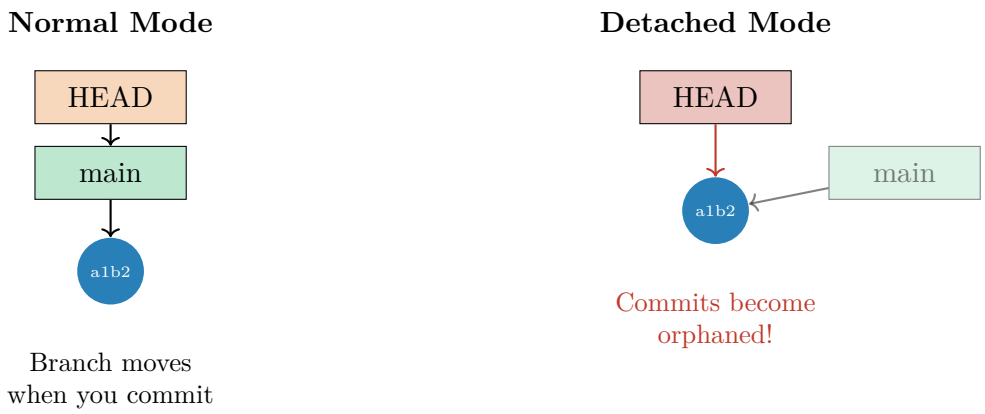


Figure 4.2: HEAD in Normal vs Detached Mode

4.3 Step 14: Enter Detached HEAD State

Let's intentionally enter detached HEAD state to understand it:

```
# First, see your commit history
git log --oneline

# Checkout a specific commit directly (use your actual hash)
git checkout a1b2c3d
```

Git gives you a helpful warning:

```
You are in 'detached HEAD' state. You can look around, make
experimental changes and commit them, and you can discard any
commits you make in this state without impacting any branches
by switching back to a branch.

If you want to create a new branch to retain commits you create,
you may do so (now or later) by using -c with the switch command.

HEAD is now at a1b2c3d Initial commit: TaskFlow project structure
```

Danger of Detached HEAD

In detached HEAD state, any commits you make aren't attached to any branch. They can be garbage collected and **permanently lost**!

When is detached HEAD useful?

- Exploring old code ("What did this look like last week?")
- Running tests on a specific version
- Quick experiments you don't need to keep

When is it dangerous?

- If you make commits you want to keep
- If you forget you're in detached state

4.4 HEAD Relatives: Referring to Previous Commits

You don't always need to use commit hashes. Git provides shortcuts to refer to commits relative to HEAD.

Reference	Meaning
`HEAD`	Current commit
`HEAD~1` or `HEAD^`	Parent of current commit (1 back)
`HEAD~2` or `HEAD^^`	Grandparent (2 commits back)
`HEAD~n`	n commits back in history
`HEAD@{1}`	Where HEAD was 1 move ago (from reflog)
`HEAD@{yesterday}`	Where HEAD was yesterday

Table 4.1: HEAD Relative References

```
# View the parent commit
git show HEAD~1

# Compare current with 2 commits ago
git diff HEAD~2

# Reset to 3 commits ago
git reset HEAD~3

# See what HEAD pointed to yesterday
git show HEAD@{yesterday}
```

Tilde vs Caret

For linear history, `~` and `^` are the same:

- `HEAD~1` = `HEAD^` = parent
- `HEAD~2` = `HEAD^^` = grandparent

The difference matters only for merge commits (which have multiple parents). We'll cover that in Chapter 5.

4.5 Step 15: Recovery with Reflog

Git's reflog (reference log) is your safety net. It records every position HEAD has been in:

```
git reflog
```

```
a1b2c3d HEAD@{0}: checkout: moving from feature/delete-task to
    a1b2c3d
h8i9j0k HEAD@{1}: commit: feat: Add task deletion functionality
d4e5f6g HEAD@{2}: checkout: moving from main to feature/delete-task
d4e5f6g HEAD@{3}: checkout: moving from feature/complete-task to main
f6g7h8i HEAD@{4}: commit: feat: Add ability to mark tasks as complete
```

Reflog: Your Safety Net

The reflog shows every HEAD movement for **90 days** by default. This means:

- Commits are almost never truly lost
- You can undo almost any mistake
- Even "deleted" branches can be recovered
- Git is more forgiving than you think

If you ever lose a commit, `git reflog` combined with `git checkout` or `git cherry-pick` can save you.

Return to safety:

```
git checkout main
# Or: git switch main
```

4.6 Chapter Summary: HEAD Mastery

Concept	Key Takeaway
Normal HEAD	Points to a branch; commits move the branch forward
Detached HEAD	Points to a commit; commits become orphaned
Entering detached	`git checkout <commit-hash>`
Exiting detached	`git checkout main` or any branch
Reflog	Records every HEAD movement for 90 days
Recovery	Use reflog to find "lost" commits
Relatives	`HEAD~n` to refer to ancestor commits

Table 4.2: HEAD Concepts Summary

Golden Rules for HEAD

1. **Always know where you are**: Check `git status` or `git branch`
2. **Avoid detached HEAD for real work**: Create a branch if you want to keep commits
3. **Don't panic about mistakes**: Reflog can recover almost anything
4. **Stay attached 99% of the time**: Normal HEAD state is the safe state

Now that you understand how to navigate Git's history and recover from mistakes, let's learn how to combine branches in Chapter 5.

5 Merging – Preserving History

Now that we have features developed on separate branches and understand how HEAD determines where commits go (from Chapter 4), we need to bring those branches together. Merging is how Git combines divergent histories.

5.1 Concept: Git's Merge Strategies

Git's Default Merge Strategy: ORT

As of Git 2.33 (August 2021), the default merge strategy is **"ort"** (Ostensibly Recursive's Twin), replacing the old "recursive" strategy.
ORT is:

- **Faster**: Especially for large repositories with many files
- **Better at detecting renames**: Handles refactoring gracefully
- **More memory-efficient**: Works well even with limited RAM
- **Produces identical results**: To the old recursive strategy in most cases

You don't need to configure anything – Git uses ORT automatically.

5.2 Step 16: Fast-Forward Merge

First, let's merge the complete-task feature into main:

```
git checkout main
git merge feature/complete-task
```

```
Updating d4e5f6g..f6g7h8i
Fast-forward
 taskflow.py | 15 +++++++++++++++
 1 file changed, 15 insertions(+)
```

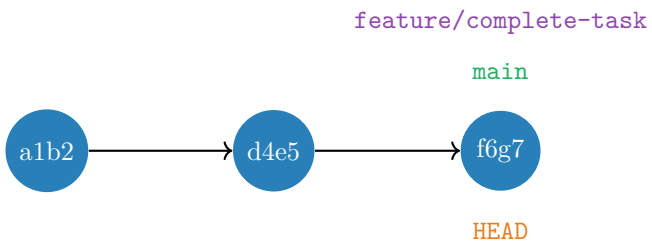


Figure 5.1: Fast-Forward Merge: main Simply Moved Forward

Fast-Forward Merge

A fast-forward merge happens when there's a **direct path** from the current branch to the target branch. Git simply moves the pointer forward – no merge commit is created. This happens because `main` was an ancestor of `feature/complete-task`. Git just "fast-forwarded" the main pointer to catch up.

Characteristics:

- No new commit created
- History remains linear
- Simple and clean

5.3 Step 17: Three-Way Merge

Now merge the delete-task feature:

```
git merge feature/delete-task
```

This time it's different:

```
Merge made by the 'ort' strategy.
 taskflow.py | 12 ++++++++++++
 1 file changed, 12 insertions(+)
```

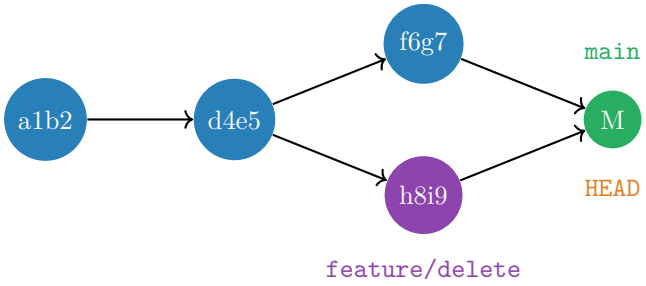


Figure 5.2: Three-Way Merge Creates a Merge Commit

Merge Commits Have Two Parents

A merge commit is special: it has **two parents**. This preserves the complete history of both branches.

The merge commit (M) has:

- **First parent**: f6g7 (from the branch you were on – main)
- **Second parent**: h8i9 (from the branch you merged – feature/delete)

This is why the history forms a diamond shape when branches merge.

6 Conflicts – When Histories Collide

Conflicts are inevitable when multiple developers (or even one developer on multiple branches) modify the same code. Learning to resolve them confidently is a critical skill.

6.1 Concept: What Causes Conflicts

Understanding Conflicts

A conflict occurs when:

1. Two branches modify the **same lines** of a file
2. Git cannot automatically determine which version to keep

Conflicts are **normal and expected**. They mean two lines of work touched the same code, and Git needs your human judgment to decide the final result.

Common scenarios that cause conflicts:

- Two people edit the same function
- One person refactors while another adds features
- Merging a long-running feature branch
- Rebasing onto a branch with conflicting changes

6.2 Types of Conflicts You'll Encounter

Conflict Taxonomy

Git conflicts fall into several categories, each requiring different resolution strategies:

Content Conflicts Two branches modify the same lines in a file (what we'll practice)

Add/Add Conflicts Both branches add a new file with the same name but different content

Modify/Delete Conflicts One branch modifies a file while another deletes it

Rename Conflicts One branch renames a file while another modifies it

Directory/File Conflicts One branch creates a directory where another has a file

Submodule Conflicts Changes to Git submodule references (advanced)

a. Content Conflicts (Most Common)

When two branches edit overlapping lines, Git cannot merge automatically.

Example: Both branches modify `list_tasks()` differently.

Resolution: Manually combine changes or choose one version.

b. Add/Add Conflicts

Both branches independently create a file with the same name.

```
# Branch A creates config.json
# Branch B creates config.json (different content)
# Merge produces: "both added" conflict
```

Resolution: Decide which version to keep or merge content manually.

c. Modify/Delete Conflicts

One branch modifies a file while another deletes it.

```
CONFLICT (modify/delete): old_feature.py deleted in feature/cleanup
and modified in feature/enhance. Version feature/enhance of
old_feature.py left in tree.
```

Resolution: Decide if the file should exist:

- Keep and merge modifications: `git add old_feature.py`
- Accept deletion: `git rm old_feature.py`

d. Rename Conflicts

File renamed in one branch but modified in another (Git usually handles this).

When it fails: Git loses track of the rename relationship.

Resolution: Manually apply changes to the renamed file.

6.3 Conflict Prevention Strategies

Minimize Conflicts Before They Happen

While conflicts are inevitable, good practices reduce their frequency:

1. **Pull frequently**: `git pull origin main` keeps your branch up-to-date
2. **Small, focused commits**: Smaller changes = fewer overlaps
3. **Communicate with team**: Coordinate who's working on what
4. **Short-lived branches**: Merge feature branches quickly (days, not weeks)
5. **Modular code**: Well-separated concerns reduce overlap
6. **Code ownership**: Assign clear responsibility for different modules

The golden rule: The longer your branch lives, the more conflicts you'll face when merging.

6.4 Step 18: Create Conflicting Branches

Let's intentionally create a conflict to learn how to resolve it. We'll create two branches that modify the same function differently.

```
git checkout -b feature/priority-colors
```

Modify `list_tasks()` to show colors:

```
def list_tasks():
    """List all tasks with color-coded priorities."""
    data = load_tasks()
    if not data["tasks"]:
        print("No tasks yet.")
        return

    print("\nYour Tasks:")
    print("-" * 50)
    for task in data["tasks"]:
        status = "[DONE]" if task["completed"] else "[    ]"
        # Color based on priority using ANSI codes
        if task["priority"] == "high":
            color = "\033[91m"  # Red
        elif task["priority"] == "medium":
            color = "\033[93m"  # Yellow
        else:
            color = "\033[92m"  # Green
        reset = "\033[0m"
        print(f"{status} #{task['id']} {color}{task['title']}{reset}"
            )
```

```
git add taskflow.py
git commit -m "feat: Add color-coded priority display"
```

Now go back to main and create a **different** version:

```
git checkout main
git checkout -b feature/priority-emoji
```

Modify `list_tasks()` to show emojis instead:

```
def list_tasks():
    """List all tasks with emoji indicators."""
    data = load_tasks()
    if not data["tasks"]:
        print("No tasks yet.")
        return

    print("\nYour Tasks:")
    print("=" * 50)
    for task in data["tasks"]:
        status = "Done" if task["completed"] else "Pending"
        # Emoji based on priority
        if task["priority"] == "high":
            emoji = "!!!"  # Urgent
        elif task["priority"] == "medium":
            emoji = "!!"   # Important
        else:
            emoji = "!"    # Normal
        print(f"[{status}] {emoji} Task #{task['id']}: {task['title
            ']}")
```

```
git add taskflow.py
git commit -m "feat: Add emoji priority indicators"
```

6.5 Step 19a: Trigger the Conflict

```
git checkout main
git merge feature/priority-colors
# Fast-forward succeeds

git merge feature/priority-emoji
```

```
Auto-merging taskflow.py
CONFLICT (content): Merge conflict in taskflow.py
Automatic merge failed; fix conflicts and then commit the result.
```

6.6 Step 19b: Inspect the Conflict

Before jumping into resolution, understand what happened:

```
# See which files are conflicted
git status

# See the conflict in context
git diff

# See what changed on each side
git log --merge --left-right --oneline

# See the common ancestor (base)
git show :1:taskflow.py   # base version
git show :2:taskflow.py   # your version (HEAD)
git show :3:taskflow.py   # their version (feature/priority-emoji)
```

⚠ Don't Panic!

Seeing conflict markers doesn't mean you did something wrong. It means Git is asking for your help to make an intelligent decision.
You can always abort:

```
git merge --abort  # Cancel the merge, return to pre-merge state
git rebase --abort # Cancel the rebase
```

6.7 Step 20: Anatomy of a Conflict

Open `taskflow.py` and find the conflict markers:

```
<<<<<<< HEAD
def list_tasks():
    """List all tasks with color-coded priorities."""
    # ... color version ...
=======
def list_tasks():
    """List all tasks with emoji indicators."""
    # ... emoji version ...
>>>>>>> feature/priority-emoji
```

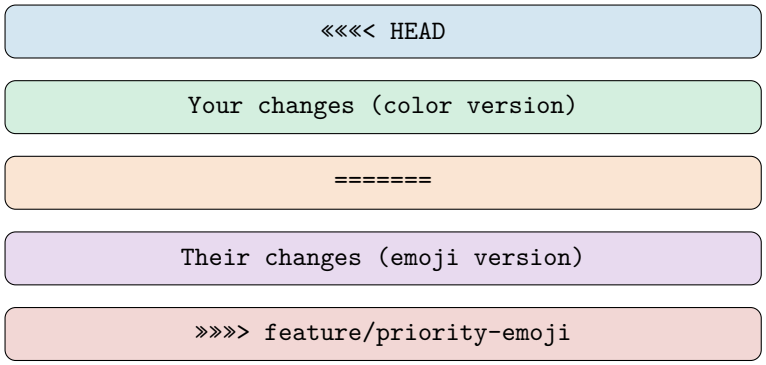


Figure 6.1: Anatomy of a Conflict Marker

Reading Conflict Markers

The markers divide the conflict into sections:

- `«««« HEAD`: Start of conflict, followed by **your** version (the branch you're on)
- `=======`: Separator between the two versions
- `»»»» branch-name`: End of conflict, preceded by **their** version (the branch being merged)

Your job is to:

1. Understand both versions
2. Decide what the final code should look like
3. Remove all conflict markers
4. Stage and commit the resolution

6.8 Step 21: Resolve the Conflict

Let's combine both features – colors *and* emojis:

==code on the following page==

Listing 6.1: list_tasks() after merge conflict resolution (v0.5.0)

```
def list_tasks():
    """Display all tasks with status and priority indicators.

    Shows tasks with:
    - Completion status (checkbox)
    - Priority indicator (symbols)
    - Color coding (ANSI colors)
    - Task ID and title

    Returns:
        None
    """
    data = load_tasks()

    if not data["tasks"]:
        print("\nNo tasks yet. Add one with: python taskflow.py add <
            title>")
        return

    print(f"\nTaskFlow v{__version__} - Your Tasks:")
    print("=" * 55)

    for task in data["tasks"]:
        # Status checkbox
        status = "[x]" if task["completed"] else "[ ]"

        # Priority indicators with colors (merged solution)
        if task["priority"] == "high":
            indicator = "!!!"
            color = "\033[91m"  # Red
        elif task["priority"] == "medium":
            indicator = "!! "
            color = "\033[93m"  # Yellow
        else:
            indicator = "!  "
            color = "\033[92m"  # Green

        reset = "\033[0m"

        # Display task with color and symbols
        print(f"{status} {indicator} {color}#{task['id']} {task['
            title']}{reset}")

    print("=" * 55)
```

Mark as resolved and complete the merge:

```
git add taskflow.py
git commit -m "merge: Combine color and emoji priority display"
```

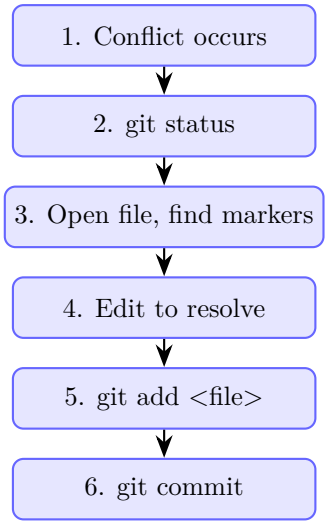


Figure 6.2: Conflict Resolution Workflow

Tools for Conflict Resolution

While you can resolve conflicts in any text editor, specialized tools make it easier:

- **VS Code**: Built-in merge conflict highlighting with "Accept Current/Incoming/Both" buttons
- **git mergetool**: Opens a configured diff tool
- **GitKraken, Sourcetree**: Visual Git clients with merge interfaces
- **IntelliJ/PyCharm**: Excellent three-way merge view

Configure your preferred tool:

```
git config --global merge.tool vscode
git config --global mergetool.vscode.cmd 'code --wait $MERGED'
```

6.9 Step 22: Test Your Resolution

⚠ Always Test After Resolving

A syntactically valid resolution might still be logically broken. The merged code must **work**.

```
# Run your program
python taskflow.py list

# Run tests if you have them
pytest tests/

# Verify the output looks correct
python taskflow.py add "Test merged feature" --priority high
python taskflow.py list  # Should show colors AND symbols
```

Why Testing Matters

Consider this scenario:

- Branch A renames function `calculate()` to `compute()`
- Branch B adds code calling `calculate()`
- Merge succeeds with no conflict!
- Code calls non-existent function → runtime error

This is called a **semantic conflict** – syntactically valid but logically broken. Git cannot detect these; only tests can.

6.10 Advanced: Conflict Resolution Strategies

When to Choose "Ours" vs "Theirs"

For simple conflicts where one side is clearly correct:

```
# Accept "our" version (current branch) for entire file
git checkout --ours taskflow.py
git add taskflow.py

# Accept "their" version (incoming branch) for entire file
git checkout --theirs taskflow.py
git add taskflow.py

# For specific files during merge
git merge -X ours feature/branch    # Prefer current branch
git merge -X theirs feature/branch  # Prefer incoming branch
```

⚠ Use Sparingly

These commands resolve *all* conflicts in a file. Only use when you're certain one side is completely correct. For most conflicts, manual resolution is safer.

Handling Binary File Conflicts

Binary files (images, PDFs, compiled code) cannot be merged:

```
# Git will ask you to choose
git checkout --ours logo.png     # Keep your version
git checkout --theirs logo.png   # Keep their version
git add logo.png
```

Best practice: Store binary files in dedicated asset management systems when possible, or use version numbers (`logo_v1.png`, `logo_v2.png`).

7 Rebase – Rewriting History

Rebase is Git's most powerful – and most dangerous – command. It lets you rewrite history to create cleaner, linear commit graphs.

7.1 Concept: Rebase vs Merge

Merge vs Rebase: Two Philosophies

Merge preserves history:

- Creates a merge commit with two parents
- Shows exactly when branches diverged and reunited
- Safe – never rewrites existing commits
- Can create "messy" graphs with many merge commits

Rebase rewrites history:

- Moves commits to a new base
- Creates new commits with different hashes
- Results in a clean, linear history
- Dangerous if used on shared commits

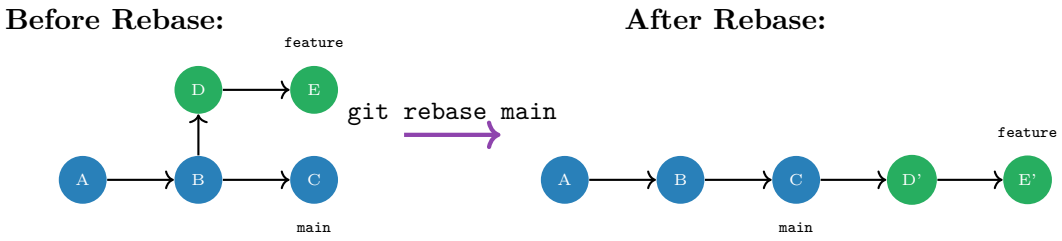


Figure 7.1: Rebase Creates New Commits (D' and E' have different hashes)

7.2 Step 23: Create a Feature to Rebase

```
git checkout -b feature/search
```

Add search functionality:

==code on the following page==

Listing 7.1: taskflow.py with search feature (v1.0.0)

```
#!/usr/bin/env python3
"""TaskFlow - A CLI Task Management Application

Author: Oussama BOUSSAHLA
Version: 1.0.0
"""

import sys
from datetime import datetime
from storage import load_tasks, save_tasks

__version__ = "1.0.0"
__author__ = "Oussama BOUSSAHLA"


def add_task(title, priority="medium"):
    """Add a new task to the task list.

    Args:
        title (str): The task description
        priority (str): Priority level - 'low', 'medium', or 'high'

    Returns:
        None
    """
    if not title or not title.strip():
        print("Error: Task title cannot be empty.")
        return

    if priority not in ('low', 'medium', 'high'):
        print(f"Error: Invalid priority '{priority}'. Using 'medium'.
            ")
        priority = 'medium'

    data = load_tasks()
    task = {
        "id": data["next_id"],
        "title": title.strip(),
        "priority": priority,
        "completed": False,
        "created": datetime.now().isoformat()
    }
    data["tasks"].append(task)
    data["next_id"] += 1
    save_tasks(data)
    print(f"Task #{task['id']} added: {title}")


def list_tasks():
    """Display all tasks with status and priority indicators.

    Returns:
        None
    """
    data = load_tasks()

    if not data["tasks"]:
```

```
57          print("\nNo tasks yet. Add one with: python taskflow.py add <
                title>")
58          return
59
60      print(f"\n{'='*60}")
61      print(f"TaskFlow v{__version__} - Your Tasks")
62      print(f"{'='*60}\n")
63
64      for task in data["tasks"]:
65          status = "[x]" if task["completed"] else "[ ]"
66
67          if task["priority"] == "high":
68              indicator = "!!!"
69              color = "\033[91m"
70          elif task["priority"] == "medium":
71              indicator = "!! "
72              color = "\033[93m"
73          else:
74              indicator = "!  "
75              color = "\033[92m"
76
77          reset = "\033[0m"
78          print(f"{status} {indicator} {color}#{task['id']:3d} {task['
                title']}{reset}")
79
80      print(f"\n{'='*60}")
81      print(f"Total: {len(data['tasks'])} tasks")
82      completed = sum(1 for t in data['tasks'] if t['completed'])
83      print(f"Completed: {completed}/{len(data['tasks'])}")
84      print(f"{'='*60}\n")
85
86
87  def complete_task(task_id):
88      """Mark a task as completed.
89
90      Args:
91          task_id (int): The ID of the task to complete
92
93      Returns:
94          None
95      """
96      data = load_tasks()
97
98      for task in data["tasks"]:
99          if task["id"] == task_id:
100             if task["completed"]:
101                 print(f"Task #{task_id} is already complete.")
102                 return
103
104             task["completed"] = True
105             task["completed_at"] = datetime.now().isoformat()
106             save_tasks(data)
107             print(f"Task #{task_id} marked as complete!")
108             return
109
110     print(f"Error: Task #{task_id} not found.")
111
112
```

```python
def delete_task(task_id):
    """Delete a task by its ID.

    Args:
        task_id (int): The ID of the task to delete

    Returns:
        None
    """
    data = load_tasks()
    original_count = len(data["tasks"])

    data["tasks"] = [t for t in data["tasks"] if t["id"] != task_id]

    if len(data["tasks"]) < original_count:
        save_tasks(data)
        print(f"Task #{task_id} deleted.")
    else:
        print(f"Error: Task #{task_id} not found.")


def search_tasks(query):
    """Search tasks by title (case-insensitive).

    Args:
        query (str): Search string to match against task titles

    Returns:
        None
    """
    if not query or not query.strip():
        print("Error: Search query cannot be empty.")
        return

    data = load_tasks()
    query_lower = query.lower()

    # Find matching tasks
    results = [t for t in data["tasks"]
               if query_lower in t["title"].lower()]

    if not results:
        print(f"\nNo tasks found matching '{query}'\n")
        return

    print(f"\n{'='*60}")
    print(f"Search Results: '{query}'")
    print(f"{'='*60}\n")

    for task in results:
        status = "Done" if task["completed"] else "Pending"
        priority = task["priority"].upper()
        print(f"  [{status}] #{task['id']:3d} {task['title']} ({
            priority})")

    print(f"\n{'='*60}")
    print(f"Found {len(results)} task(s)")
    print(f"{'='*60}\n")
```

```
170
171
172 def show_help():
173     """Display help information.
174
175     Returns:
176         None
177     """
178     print(f"TaskFlow v{__version__}")
179     print("Usage: python taskflow.py <command> [args]")
180     print("\nCommands:")
181     print("  add <title>     Add a new task")
182     print("  list            List all tasks")
183     print("  done <id>       Mark task as complete")
184     print("  delete <id>     Delete a task")
185     print("  search <query>  Search tasks by title")
186     print("  help            Show this help")
187
188
189 def main():
190     """Main entry point - routes commands to functions.
191
192     Returns:
193         None
194     """
195     if len(sys.argv) < 2:
196         show_help()
197         return
198
199     command = sys.argv[1].lower()
200
201     if command == "add" and len(sys.argv) >= 3:
202         title = " ".join(sys.argv[2:])
203         add_task(title)
204     elif command == "list":
205         list_tasks()
206     elif command == "done" and len(sys.argv) >= 3:
207         try:
208             task_id = int(sys.argv[2])
209             complete_task(task_id)
210         except ValueError:
211             print("Error: Task ID must be a number.")
212     elif command == "delete" and len(sys.argv) >= 3:
213         try:
214             task_id = int(sys.argv[2])
215             delete_task(task_id)
216         except ValueError:
217             print("Error: Task ID must be a number.")
218     elif command == "search" and len(sys.argv) >= 3:
219         query = " ".join(sys.argv[2:])
220         search_tasks(query)
221     elif command == "help":
222         show_help()
223     else:
224         print(f"Unknown command: {command}")
225         print("Run 'python taskflow.py help' for usage.")
226
227
```

```
228 if __name__ == "__main__":
229     main()
```

```
git add taskflow.py
git commit -m "feat: Add task search functionality"
```

Meanwhile, simulate work happening on main:

```
git checkout main
echo "tests.py" >> .gitignore
git add .gitignore
git commit -m "chore: Add tests.py to .gitignore"
```

7.3 Step 24: Rebase the Feature Branch

```
git checkout feature/search
git rebase main
```

```
Successfully rebased and updated refs/heads/feature/search.
```

Prediction Challenge

What happened to the original search commit?
Answer:

- The old commit is **orphaned** (still exists but unreachable from any branch)
- A **new commit** was created with the same changes but a different hash
- Why? Because the commit's parent changed (now it's based on the gitignore commit)

The old commit will eventually be garbage collected.

7.4 When to Use Each Approach

Situation	Use Merge	Use Rebase
Branch has been pushed/shared	✓	
Personal/local feature branch		✓
Want clean linear history		✓
Need to preserve exact history	✓	
Long-running feature with many commits	✓	
Quick cleanup before PR		✓
Collaborative branch	✓	

Table 7.1: When to Use Merge vs Rebase

The Golden Rule of Rebase

Never rebase commits that have been pushed to a shared repository.
Why? Because rebase rewrites history – it creates new commits with new hashes. If others have based their work on the old commits, their branches become orphaned.
Safe to rebase: Local branches that only you have seen
Never rebase: Anything on `main`, `develop`, or shared feature branches

8 Reset, Checkout, Revert – Undoing Changes

Mistakes happen. Git provides multiple ways to undo them, each suited to different situations.

8.1 Concept: Three Ways to Undo

The Undo Commands

Git has three main ways to undo changes:

1. **git restore**: Discards changes in working directory or staging area
2. **git reset**: Moves branch pointers backward (rewrites history)
3. **git revert**: Creates a new commit that undoes changes (safe for shared branches)

8.2 Step 25: Make a Mistake

Let's intentionally add buggy code:

```
git checkout main
```

Add this buggy function to `taskflow.py`:

```
def buggy_function():
    """This function has a critical bug!"""
    return 1 / 0  # Division by zero - will crash!
```

```
git add taskflow.py
git commit -m "feat: Add new calculation function (BUGGY)"
```

8.3 Step 26: Undo with Reset (For Unpushed Commits)

Since we haven't pushed this commit, we can safely reset:

```
# Undo commit but keep changes staged
git reset --soft HEAD~1

# Undo commit and unstage (but keep working directory)
git reset --mixed HEAD~1  # or just 'git reset HEAD~1'

# Undo commit AND discard all changes (DANGEROUS!)
git reset --hard HEAD~1
```

Reset Mode	Branch Pointer	Staging Area	Working Directory
`-soft`	Moves back	*Unchanged*	*Unchanged*
`-mixed` (default)	Moves back	Cleared	*Unchanged*
`-hard`	Moves back	Cleared	**Cleared!**

Table 8.1: Reset Modes and What They Affect

⚠ Reset –hard is Destructive

`git reset -hard` will permanently discard your working directory changes. There is no undo for this operation (except through reflog if you had committed first).
Always double-check before using `-hard`!

8.4 Step 27: Undo with Revert (For Pushed Commits)

If the buggy commit was already pushed, use revert instead:

```
git revert HEAD
```

This creates a **new commit** that undoes the changes. History is preserved:

```
[main abc123] Revert "feat: Add new calculation function (BUGGY)"
 1 file changed, 4 deletions(-)
```

The original buggy commit still exists in history, but its changes are negated by the revert commit.

8.5 Decision Tree: Which Undo Command?

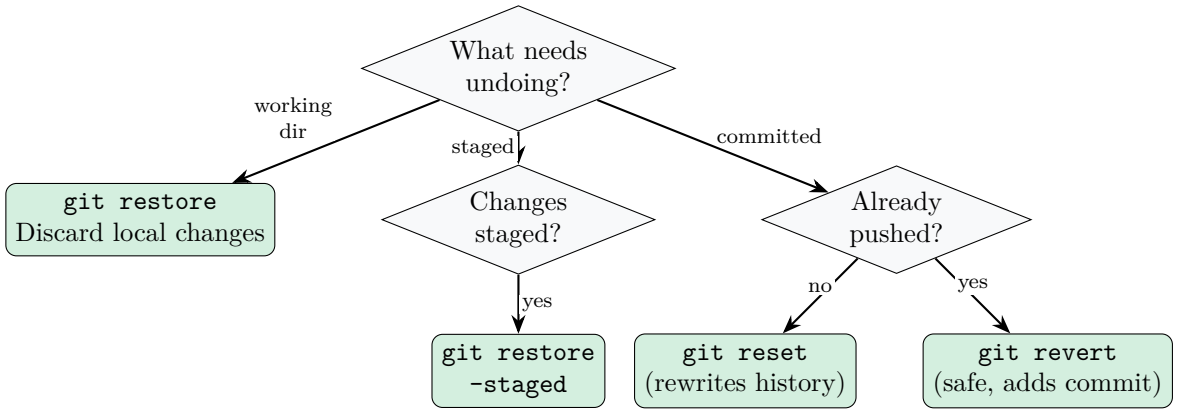


Figure 8.1: Decision Tree: Choosing the Right Undo Command

Quick Reference for Undoing

- **Discard unstaged changes**: `git restore <file>`
- **Unstage a file**: `git restore -staged <file>`
- **Undo last commit (keep changes)**: `git reset -soft HEAD~1`
- **Undo last commit (discard changes)**: `git reset -hard HEAD~1`
- **Undo a pushed commit**: `git revert <commit-hash>`
- **Recover lost commit**: `git reflog` then `git checkout <hash>`

9 Tags – Marking Releases

Tags are permanent markers for important points in history – typically releases.

9.1 Concept: Tags vs Branches

Tags vs Branches

Both are references to commits, but they behave differently:
Branches:

- Move forward when you commit
- Designed for ongoing development
- Mutable (point changes over time)

Tags:

- Never move (permanent marker)
- Designed for releases and milestones
- Immutable (point never changes)

9.2 Step 28: Create Release Tags

Listing 9.1: export_tasks() function for Chapter 14

```
def export_tasks(filename="tasks_export.txt"):
    """Export all tasks to a text file.

    Creates a formatted text file with all tasks, including:
    - Export timestamp
    - Task status, ID, title, and priority
    - Summary statistics

    Args:
        filename (str): Name of the export file (default:
            tasks_export.txt)

    Returns:
        None
    """
    data = load_tasks()

    try:
        with open(filename, 'w', encoding='utf-8') as f:
            # Header
            f.write("="*60 + "\n")
            f.write(f"TaskFlow Export - {datetime.now().strftime('%Y
                -%m-%d %H:%M:%S')}\n")
            f.write(f"TaskFlow v{__version__}\n")
            f.write("="*60 + "\n\n")

            if not data["tasks"]:
```

```
                f.write("No tasks to export.\n")
            else:
                # Tasks
                for task in data["tasks"]:
                    status = "[x]" if task["completed"] else "[ ]"
                    priority = task["priority"].upper()
                    f.write(f"{status} #{task['id']:3d} {task['title
                        ']} ({priority})\n")

                    # Additional details
                    created = datetime.fromisoformat(task["created"])
                        .strftime('%Y-%m-%d %H:%M')
                    f.write(f"      Created: {created}\n")

                    if task["completed"] and "completed_at" in task:
                        completed = datetime.fromisoformat(task["
                            completed_at"]).strftime('%Y-%m-%d %H:%M')
                        f.write(f"      Completed: {completed}\n")

                    f.write("\n")

                # Summary
                f.write("="*60 + "\n")
                f.write(f"Total tasks: {len(data['tasks'])}\n")
                completed = sum(1 for t in data['tasks'] if t['
                    completed'])
                f.write(f"Completed: {completed}\n")
                f.write(f"Pending: {len(data['tasks']) - completed}\n
                    ")
                f.write("="*60 + "\n")

        print(f"Exported {len(data['tasks'])} task(s) to {filename}")

    except IOError as e:
        print(f"Error: Could not write to file '{filename}': {e}")
```

Listing 9.2: Updated main() to include export command

```
def main():
    """Main entry point - parse arguments and route to functions."""
    if len(sys.argv) < 2:
        show_help()
        return

    command = sys.argv[1].lower()

    if command == "add" and len(sys.argv) >= 3:
        title = " ".join(sys.argv[2:])
        add_task(title)
    elif command == "list":
        list_tasks()
    elif command == "done" and len(sys.argv) >= 3:
        try:
            task_id = int(sys.argv[2])
            complete_task(task_id)
        except ValueError:
            print("Error: Task ID must be a number.")
```

```
    elif command == "delete" and len(sys.argv) >= 3:
        try:
            task_id = int(sys.argv[2])
            delete_task(task_id)
        except ValueError:
            print("Error: Task ID must be a number.")
    elif command == "search" and len(sys.argv) >= 3:
        query = " ".join(sys.argv[2:])
        search_tasks(query)
    elif command == "export":
        filename = sys.argv[2] if len(sys.argv) >= 3 else "
            tasks_export.txt"
        export_tasks(filename)
    elif command == "help":
        show_help()
    else:
        print(f"Unknown command: {command}")
        print("Run 'python taskflow.py help' for usage.")
```

Listing 9.3: Update version number to 1.1.0

```
__version__ = "1.1.0"
```

View your tags:

```
git tag           # List all tags
git show v1.0.0   # Show tag details and associated commit
```

9.3 Step 29: Return to Tagged Versions

Tags make it easy to return to any release:

```
# View code at a specific release
git checkout v1.0.0

# Create a hotfix branch from an old release
git checkout -b hotfix/v1.0.1 v1.0.0
```

Semantic Versioning

Use semantic versioning (SemVer) for your tags: `vMAJOR.MINOR.PATCH`

- **MAJOR**: Breaking changes (v1.0.0 → v2.0.0)
- **MINOR**: New features, backward compatible (v1.0.0 → v1.1.0)
- **PATCH**: Bug fixes (v1.0.0 → v1.0.1)

Always use annotated tags for releases – they store who created the tag, when, and why.

10 GitHub – Cloud Collaboration

GitHub is where Git meets the internet. It's the world's largest code hosting platform, used by millions of developers and companies.

10.1 Understanding GitHub

What is GitHub?

GitHub is a web-based platform that provides:

- **Remote repository hosting**: Store your Git repositories in the cloud
- **Collaboration tools**: Pull requests, code review, issues
- **Social features**: Follow developers, star projects, contribute to open source
- **CI/CD**: GitHub Actions for automated testing and deployment
- **Project management**: Issues, projects, milestones

GitHub is **not** Git – it's a service built on top of Git that adds collaboration features.

10.2 Creating a GitHub Repository

1. Go to `https://github.com` and sign in (or create an account)
2. Click the "+" button in the top right → "New repository"
3. Configure your repository:
   - **Repository name**: taskflow
   - **Description**: A CLI task manager built while learning Git
   - **Visibility**: Public (for learning) or Private
   - **Initialize**: Leave empty (we have an existing repo)
4. Click "Create repository"

10.3 The main vs master Problem

Understanding the Issue

⚠ Branch Naming Incompatibility

One of the most frustrating issues for Git beginners is the mismatch between:

- **Local Git**: Creates `master` branch by default (older versions)
- **GitHub**: Creates `main` branch by default (since 2020)

This causes common errors like:

```
error: src refspec main does not exist
error: failed to push some refs to 'github.com/user/repo.git'
```

Why this happened: In 2020, GitHub changed the default branch name from `master`

to `main` to use more inclusive language. Git followed suit, but many local installations still default to `master`.

Checking Your Default Branch Name

```
# Check your current branch
git branch
# Output: * master  (or * main)

# Check Git's default branch name setting
git config --global init.defaultBranch
# Output: main (or empty if not set)

# Check your Git version
git --version
# Versions 2.28+ support init.defaultBranch
```

Solution 1: Change Local Git Default (Recommended)

The best solution is to configure your local Git to use `main` by default:

```
# Set main as default for ALL new repositories
git config --global init.defaultBranch main

# Verify it's set
git config --global init.defaultBranch
# Output: main

# From now on, all new repos will use 'main'
mkdir new-project
cd new-project
git init
git branch
# Output: (no branches yet, but first commit creates 'main')
```

Do This Once, Solve It Forever

Run this command on every computer where you use Git:
`git config --global init.defaultBranch main`
This is a **one-time configuration** that fixes the issue for all future projects.

Solution 2: Rename Existing master to main

If you already have a repository with `master`, rename it:

```
# Make sure you're on master
git checkout master

# Rename master to main
git branch -m master main

# Verify
git branch
```

```
# Output: * main

# If you've already pushed to GitHub, update the remote
git push -u origin main

# On GitHub: Go to Settings > Branches > Default branch
# Change from master to main

# Delete old master branch on GitHub
git push origin --delete master
```

Solution 3: Rename main to master (Not Recommended)

If you prefer to use `master` (to match older tutorials), you can:

```
# Clone a GitHub repo that uses 'main'
git clone https://github.com/user/repo.git
cd repo

# Rename main to master
git branch -m main master

# Set it as default remote
git push -u origin master

# On GitHub: Settings > Branches > change default to master
# Then delete the 'main' branch on GitHub
git push origin --delete main
```

⚠ Why main is Better

We recommend using `main` because:

- It's the GitHub default (easier collaboration)
- It's the Git default (versions 2.28+)
- Most new projects use it
- It's more inclusive language
- Less configuration needed

Stick with `master` only if:

- You're working with legacy codebases
- Your organization requires it
- You're following old tutorials

Common Scenarios and Fixes

Scenario 1: Pushing to GitHub for First Time

```
# You created a local repo with 'master'
git init
git add .
git commit -m "Initial commit"

# Created GitHub repo (which has 'main')
# Trying to push fails:
git push -u origin main
# error: src refspec main does not exist

# FIX: Rename your branch to main
git branch -m master main
git push -u origin main
# Success!
```

Scenario 2: Cloned Repo, Local Shows Different Branch

```
# Clone a repo
git clone https://github.com/user/repo.git
cd repo

# Check your branch
git branch
# Output: * main  (Good! You're on main)

# If it shows master instead:
git checkout main  # Switch to main
git branch -d master  # Delete local master
```

Scenario 3: Mixed Branches (main and master exist)

```
# You have both branches somehow
git branch
# * main
#   master

# Check which one is actually being used
git log main --oneline -3
git log master --online -3

# If they're the same, delete master:
git branch -d master

# If they're different, you need to decide:
# Option A: Merge master into main
git checkout main
git merge master
git branch -d master
```

```
# Option B: Use master, delete main
git checkout master
git branch -d main
```

Preventing Future Issues

Best Practices for Branch Names

1. Set global default immediately:
`git config --global init.defaultBranch main`
2. Always check before pushing:
`git branch  # See what you're on locally`
`git push -u origin main  # Match GitHub's default`
3. When creating GitHub repos:
- GitHub now defaults to `main`
- Don't initialize with README if you have local commits
- Or, initialize and clone it (GitHub creates `main`)

4. When cloning repos:
- Git automatically sets up the right branch
- Just use whatever branch GitHub has

Team Collaboration: Agreeing on Branch Names

Team Standards

When working in a team:
Option A: Everyone uses main (Recommended)
- All team members run: `git config -global init.defaultBranch main`
- GitHub repos use `main`
- Documentation references `main`
- Consistent and modern

Option B: Legacy projects stick with master
- Document this decision
- Change GitHub default to `master`
- Team members configure accordingly
- Only if absolutely necessary

Communication is key:
- Document in README: "This project uses `main`"
- Include in setup instructions
- Add to CONTRIBUTING.md

Understanding the Error Messages

Error	Cause & Fix
`src refspec main does not exist`	You're trying to push `main` but your local branch is `master`. Fix: `git branch -m master main`
`failed to push some refs`	Branch mismatch or diverged history. Check: `git branch` then match the remote
`refusing to merge unrelated histories`	GitHub repo has commits (like README) and your local has different commits. Fix: `git pull origin main -allow-unrelated-histories`
`fatal: branch 'main' does not exist`	You're on `master` trying to checkout `main`. Fix: `git branch -m master main`

Table 10.1: Common Branch Name Errors

The Complete Setup Checklist

Follow this checklist to avoid branch name issues:

1. **Configure Git globally** (do once):

```
git config --global init.defaultBranch main
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

2. **For new projects**:

```
# Create repo on GitHub first (it will use 'main')
# Then clone it
git clone https://github.com/username/repo.git
cd repo
# You're automatically on 'main' - no issues!
```

3. **Or, create locally first**:

```
mkdir project
cd project
git init  # Creates with 'main' (if configured)
git add .
git commit -m "Initial commit"
# Create empty repo on GitHub (no README)
git remote add origin https://github.com/username/repo.git
git push -u origin main
```

4. **Verify setup**:

```
git branch  # Should show: * main
git remote -v  # Should show origin pointing to GitHub
git log --oneline -1  # Should show your commits
```

Quick Reference

Default branch configuration:

```
# See current setting
git config --global init.defaultBranch

# Set to 'main' (recommended)
git config --global init.defaultBranch main

# Rename existing branch
git branch -m old-name new-name

# Push and set upstream
git push -u origin main
```

Historical Context

Why the Change Happened

Timeline:

- **Pre-2020**: Git used `master` by default
- **June 2020**: GitHub announces `main` as new default
- **October 2020**: GitHub makes `main` default for new repos
- **Git 2.28 (July 2020)**: Git adds `init.defaultBranch` config
- **2021-2024**: Most projects migrate to `main`
- **Today**: `main` is standard for new projects

Technical note: The branch name is arbitrary. Git doesn't care if you use `main`, `master`, `trunk`, or `bananas`. The only issue is consistency between local and remote.

What Real Projects Do

Open source projects:

- New projects: 95%+ use `main`
- Old projects: Many migrated to `main` (Python, Django, React)
- Legacy projects: Some still use `master`

Companies:

- Most new codebases use `main`
- Large companies (Google, Microsoft, etc.) migrated
- Some legacy systems still use `master`

Your strategy: Use `main` for new projects, follow existing projects' conventions for contributions.

10.4 Step 30: Connect Local to Remote

```
# Add the remote repository
git remote add origin https://github.com/YOUR_USERNAME/taskflow.git

# Verify the remote was added
git remote -v

# Push your main branch (and set it to track the remote)
git push -u origin main

# Push all branches
git push --all origin

# Push all tags
git push --tags
```

Understanding Remotes

A **remote** is a bookmark to another copy of your repository, usually on a server.

- `origin`: The conventional name for your primary remote
- `upstream`: Often used for the original repo when you've forked
- You can have multiple remotes (e.g., GitHub and GitLab simultaneously)

When you run `git push origin main`, you're saying: "Send my `main` branch to the remote called `origin`."

10.5 Step 31: The Pull Request Workflow

Pull Requests (PRs) are GitHub's mechanism for code review and collaboration.

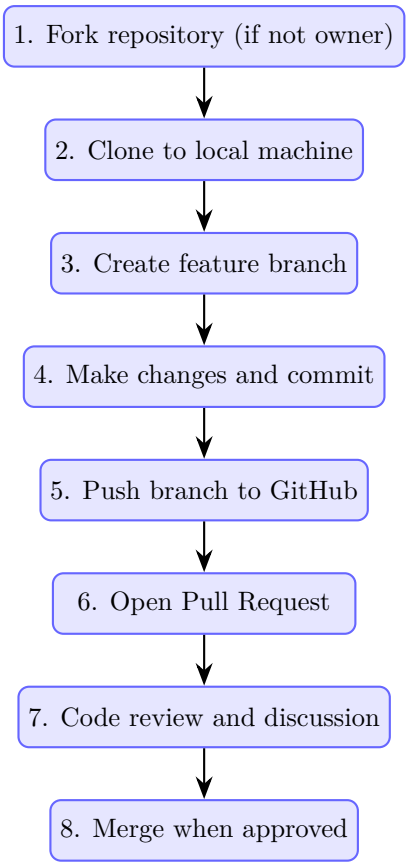


Figure 10.1: The Pull Request Workflow

10.6 Creating a Pull Request

1. Push your feature branch to GitHub:

```
git checkout feature/search
git push origin feature/search
```

2. Go to your repository on GitHub
3. You'll see a banner: "feature/search had recent pushes" → Click "Compare & pull request"
4. Fill out the PR form:
   - **Title**: Clear, descriptive title (e.g., "Add task search functionality")
   - **Description**: Explain what the PR does and why
   - **Reviewers**: Request reviews from teammates
   - **Labels**: Add relevant labels (feature, bug, etc.)
5. Click "Create pull request"

Writing Good PR Descriptions

A good PR description includes:

- **What**: Summary of changes
- **Why**: Motivation/problem being solved
- **How**: High-level approach
- **Testing**: How to test the changes
- **Screenshots**: For UI changes

Example:

```
## What
Adds search functionality to find tasks by title.

## Why
Users with many tasks need a way to quickly find specific ones.

## How
- Added `search_tasks()` function with case-insensitive matching
- Added "search" command to CLI

## Testing
python taskflow.py add "Buy groceries"
python taskflow.py add "Buy birthday gift"
python taskflow.py search "buy"
```

10.7 GitHub Issues and Project Management

GitHub Issues

Issues are GitHub's way to track bugs, features, and tasks:

- **Bug reports**: Document problems with steps to reproduce
- **Feature requests**: Propose new functionality
- **Tasks**: Track work items
- **Discussions**: Have conversations about the project

Issues can be:

- Assigned to team members
- Labeled (bug, enhancement, documentation, etc.)
- Added to milestones
- Linked to pull requests
- Automatically closed when PRs merge

To automatically close an issue when a PR merges, include in your commit message or PR description:

```
Fixes #123
Closes #456
Resolves #789
```

10.8 GitHub Actions: CI/CD

GitHub Actions lets you automate testing and deployment.

Create `.github/workflows/test.yml`:

Listing 10.1: GitHub Actions workflow for testing

```
name: Test TaskFlow

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        python-version: ['3.9', '3.10', '3.11']

    steps:
    - uses: actions/checkout@v4

    - name: Set up Python ${{ matrix.python-version }}
      uses: actions/setup-python@v5
      with:
        python-version: ${{ matrix.python-version }}

    - name: Install dependencies
      run: |
        python -m pip install --upgrade pip
        pip install pytest

    - name: Run tests
      run: |
        python -m pytest tests/ -v

    - name: Check syntax
      run: |
        python -m py_compile taskflow.py storage.py
```

```
mkdir -p .github/workflows
git add .github/workflows/test.yml
git commit -m "ci: Add GitHub Actions workflow for testing"
git push origin main
```

Now every push and pull request will automatically run your tests!

10.9 Key GitHub Features Summary

Feature	Purpose
Repositories	Host your Git repositories
Pull Requests	Propose, review, and merge changes
Issues	Track bugs, features, and tasks
Actions	Automated CI/CD workflows
Projects	Kanban boards for project management
Wiki	Documentation hosting
Releases	Package and distribute versions
Discussions	Community Q&A and conversations
Security	Vulnerability scanning, secret detection

Table 10.2: Essential GitHub Features

11 GitLab – The DevOps Platform

GitLab is GitHub's main competitor, offering similar features with some key differences.

11.1 GitLab vs GitHub

Key Differences

Feature	GitHub	GitLab
Merge terminology	Pull Request	Merge Request
CI/CD	GitHub Actions (YAML)	GitLab CI/CD (YAML)
Self-hosting	Enterprise only ($$$)	Free (Community Edition)
Container registry	GitHub Packages	Built-in registry
Issue boards	Projects	Built-in Kanban
Built-in IDE	Codespaces	Web IDE

When to Choose GitLab

GitLab is particularly popular when:

- You need **self-hosting** (many enterprises require on-premises solutions)
- You want **integrated CI/CD** without external services
- You need **built-in container registry**
- Your organization prefers an **all-in-one DevOps platform**

Many companies use both: GitHub for open-source projects, GitLab for internal code.

11.2 Step 32: GitLab Workflow

The workflow is nearly identical to GitHub:

```
# Add GitLab as a remote (you can have multiple remotes)
git remote add gitlab https://gitlab.com/YOUR_USERNAME/taskflow.git

# Push to GitLab
git push gitlab main

# Or push to both
git push origin main
git push gitlab main
```

11.3 GitLab CI/CD with .gitlab-ci.yml

GitLab's CI/CD is configured with a single file in your repository root.

Create `.gitlab-ci.yml`:

Listing 11.1: GitLab CI/CD configuration

```yaml
# Define the stages of your pipeline
stages:
  - test
  - build
  - deploy

# Variables available to all jobs
variables:
  PYTHON_VERSION: "3.11"

# Cache pip downloads between jobs
cache:
  paths:
    - .cache/pip

# Test stage - runs on every commit
test:
  stage: test
  image: python:${PYTHON_VERSION}
  before_script:
    - pip install pytest
  script:
    - python -m pytest tests/ -v --tb=short
    - python -m py_compile taskflow.py storage.py
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == "main"

# Build stage - create distributable package
build:
  stage: build
  image: python:${PYTHON_VERSION}
  script:
    - pip install build
    - python -m build
  artifacts:
    paths:
      - dist/
    expire_in: 1 week
  only:
    - main
    - tags

# Deploy stage - only on tags (releases)
deploy:
  stage: deploy
  image: python:${PYTHON_VERSION}
  script:
    - pip install twine
    - echo "Would deploy to PyPI here"
    # - twine upload dist/*  # Uncomment when ready
  only:
    - tags
  when: manual  # Require manual trigger
```

```
git add .gitlab-ci.yml
git commit -m "ci: Add GitLab CI/CD configuration"
git push gitlab main
```

11.4 GitLab Merge Requests

GitLab's "Merge Requests" are equivalent to GitHub's "Pull Requests":

1. Push your feature branch: `git push gitlab feature/search`
2. Go to your GitLab project
3. Click "Create merge request"
4. Fill in the description and assign reviewers
5. After approval, merge using one of:
   - **Merge commit**: Creates a merge commit (preserves history)
   - **Squash and merge**: Combines all commits into one
   - **Fast-forward merge**: Linear history if possible

GitLab Unique Features

GitLab offers some features not in GitHub:

- **Built-in Container Registry**: Push Docker images directly
- **Auto DevOps**: Automatic CI/CD pipeline detection
- **Merge Trains**: Queue merge requests to test combinations
- **Feature Flags**: Built-in feature toggle management
- **Requirements Management**: Track product requirements

12 The Complete TaskFlow Project

Let's review what we've built and see the final project structure.

12.1 Final Project Structure

```
taskflow/
|-- taskflow.py           # Main CLI application
|-- storage.py            # Data persistence module
|-- tasks.json            # Task data (gitignored)
|-- .gitignore            # Ignored files
|-- .github/
|   +-- workflows/
|       +-- test.yml      # GitHub Actions CI
|-- .gitlab-ci.yml        # GitLab CI configuration
|-- README.md             # Project documentation
+-- tests/
    +-- test_taskflow.py # Unit tests
```

12.2 Complete taskflow.py

Listing 12.1: Complete taskflow.py - Final Version

```
#!/usr/bin/env python3
"""TaskFlow - A CLI Task Management Application

A complete task manager built while learning Git.
Demonstrates branching, merging, and collaboration workflows.

Author: Oussama BOUSSAHLA
Version: 1.1.0

Usage:
    python taskflow.py <command> [arguments]

Commands:
    add <title> [--priority high|medium|low]
        Add a new task with optional priority

    list
        List all tasks with status and priority indicators

    done <id>
        Mark a task as complete

    delete <id>
        Delete a task permanently

    search <query>
        Search tasks by title (case-insensitive)

    export [filename]
        Export all tasks to a text file

    help
        Show this help message

Examples:
    python taskflow.py add "Review pull request" --priority high
    python taskflow.py list
    python taskflow.py done 3
    python taskflow.py search "meeting"
    python taskflow.py export tasks_backup.txt
"""

import sys
from datetime import datetime
from storage import load_tasks, save_tasks

__version__ = "1.1.0"
__author__ = "Oussama BOUSSAHLA"


def add_task(title, priority="medium"):
    """Add a new task to the task list.

    Args:
```

```
55          title (str): The task description
56          priority (str): Priority level - 'low', 'medium', or 'high'
57
58      Returns:
59          None
60      """
61      if not title or not title.strip():
62          print("Error: Task title cannot be empty.")
63          return
64
65      if priority not in ('low', 'medium', 'high'):
66          print(f"Error: Invalid priority '{priority}'. Using 'medium'.
                ")
67          priority = 'medium'
68
69      data = load_tasks()
70      task = {
71          "id": data["next_id"],
72          "title": title.strip(),
73          "priority": priority,
74          "completed": False,
75          "created": datetime.now().isoformat()
76      }
77      data["tasks"].append(task)
78      data["next_id"] += 1
79      save_tasks(data)
80      print(f"Task #{task['id']} added: {title}")
81
82
83  def list_tasks():
84      """Display all tasks with status and priority indicators.
85
86      Shows tasks with:
87      - Completion status (checkbox)
88      - Priority indicator (symbols)
89      - Color coding (ANSI colors)
90      - Task ID and title
91
92      Returns:
93          None
94      """
95      data = load_tasks()
96
97      if not data["tasks"]:
98          print("\nNo tasks yet. Add one with: python taskflow.py add <
                title>")
99          return
100
101     print(f"\n{'='*60}")
102     print(f"TaskFlow v{__version__} - Your Tasks")
103     print(f"{'='*60}\n")
104
105     for task in data["tasks"]:
106         # Status checkbox
107         status = "[x]" if task["completed"] else "[ ]"
108
109         # Priority indicators with colors
110         if task["priority"] == "high":
```

```
            indicator = "!!!"
            color = "\033[91m"  # Red
        elif task["priority"] == "medium":
            indicator = "!! "
            color = "\033[93m"  # Yellow
        else:
            indicator = "!  "
            color = "\033[92m"  # Green

        reset = "\033[0m"

        # Display task
        print(f"{status} {indicator} {color}#{task['id']:3d} {task['
            title']}{reset}")

    print(f"\n{'='*60}")
    print(f"Total: {len(data['tasks'])} tasks")
    completed = sum(1 for t in data['tasks'] if t['completed'])
    print(f"Completed: {completed}/{len(data['tasks'])}")
    print(f"{'='*60}\n")


def complete_task(task_id):
    """Mark a task as completed.

    Args:
        task_id (int): The ID of the task to complete

    Returns:
        None
    """
    data = load_tasks()

    for task in data["tasks"]:
        if task["id"] == task_id:
            if task["completed"]:
                print(f"Task #{task_id} is already complete.")
                return

            task["completed"] = True
            task["completed_at"] = datetime.now().isoformat()
            save_tasks(data)
            print(f"Task #{task_id} marked as complete!")
            return

    print(f"Error: Task #{task_id} not found.")


def delete_task(task_id):
    """Delete a task by its ID.

    Args:
        task_id (int): The ID of the task to delete

    Returns:
        None
    """
    data = load_tasks()
```

```
    original_count = len(data["tasks"])

    # Filter out the task with the given ID
    data["tasks"] = [t for t in data["tasks"] if t["id"] != task_id]

    if len(data["tasks"]) < original_count:
        save_tasks(data)
        print(f"Task #{task_id} deleted.")
    else:
        print(f"Error: Task #{task_id} not found.")


def search_tasks(query):
    """Search tasks by title (case-insensitive).

    Args:
        query (str): Search string to match against task titles

    Returns:
        None
    """
    if not query or not query.strip():
        print("Error: Search query cannot be empty.")
        return

    data = load_tasks()
    query_lower = query.lower()

    # Find matching tasks
    results = [t for t in data["tasks"]
               if query_lower in t["title"].lower()]

    if not results:
        print(f"\nNo tasks found matching '{query}'\n")
        return

    print(f"\n{'='*60}")
    print(f"Search Results: '{query}'")
    print(f"{'='*60}\n")

    for task in results:
        status = "Done" if task["completed"] else "Pending"
        priority = task["priority"].upper()
        print(f"  [{status}] #{task['id']:3d} {task['title']} ({
            priority})")

    print(f"\n{'='*60}")
    print(f"Found {len(results)} task(s)")
    print(f"{'='*60}\n")


def export_tasks(filename="tasks_export.txt"):
    """Export all tasks to a text file.

    Creates a formatted text file with all tasks, including:
    - Export timestamp
    - Task status, ID, title, and priority
    - Summary statistics
```

```
225
226     Args:
227         filename (str): Name of the export file (default:
                tasks_export.txt)
228
229     Returns:
230         None
231     """
232     data = load_tasks()
233
234     try:
235         with open(filename, 'w', encoding='utf-8') as f:
236             # Header
237             f.write("="*60 + "\n")
238             f.write(f"TaskFlow Export - {datetime.now().strftime('%Y
                    -%m-%d %H:%M:%S')}\n")
239             f.write(f"TaskFlow v{__version__}\n")
240             f.write("="*60 + "\n\n")
241
242             if not data["tasks"]:
243                 f.write("No tasks to export.\n")
244             else:
245                 # Tasks
246                 for task in data["tasks"]:
247                     status = "[x]" if task["completed"] else "[ ]"
248                     priority = task["priority"].upper()
249                     f.write(f"{status} #{task['id']:3d} {task['title
                            ']} ({priority})\n")
250
251                     # Additional details
252                     created = datetime.fromisoformat(task["created"])
                            .strftime('%Y-%m-%d %H:%M')
253                     f.write(f"      Created: {created}\n")
254
255                     if task["completed"] and "completed_at" in task:
256                         completed = datetime.fromisoformat(task["
                                completed_at"]).strftime('%Y-%m-%d %H:%M')
257                         f.write(f"      Completed: {completed}\n")
258
259                     f.write("\n")
260
261                 # Summary
262                 f.write("="*60 + "\n")
263                 f.write(f"Total tasks: {len(data['tasks'])}\n")
264                 completed = sum(1 for t in data['tasks'] if t['
                        completed'])
265                 f.write(f"Completed: {completed}\n")
266                 f.write(f"Pending: {len(data['tasks']) - completed}\n
                        ")
267                 f.write("="*60 + "\n")
268
269         print(f"Exported {len(data['tasks'])} task(s) to {filename}")
270
271     except IOError as e:
272         print(f"Error: Could not write to file '{filename}': {e}")
273
274
275 def show_help():
```

```
    """Display detailed help information.

    Shows all available commands with descriptions and examples.

    Returns:
        None
    """
    help_text = f"""
{'='*60}
TaskFlow v{__version__} - CLI Task Manager
{'='*60}

COMMANDS:

  add <title> [--priority high|medium|low]
      Add a new task with optional priority

      Examples:
        python taskflow.py add "Review PR #42"
        python taskflow.py add "Fix bug" --priority high

  list
      Display all tasks with status and priority

      Example:
        python taskflow.py list

  done <id>
      Mark a task as complete

      Example:
        python taskflow.py done 3

  delete <id>
      Permanently delete a task

      Example:
        python taskflow.py delete 5

  search <query>
      Search tasks by title (case-insensitive)

      Example:
        python taskflow.py search "meeting"

  export [filename]
      Export all tasks to a text file

      Examples:
        python taskflow.py export
        python taskflow.py export backup.txt

  help
      Show this help message

PRIORITY LEVELS:

  high   - Urgent tasks (red, !!!)
```

```
  medium - Important tasks (yellow, !!)
  low    - Normal tasks (green, !)

TIPS:

  * Use quotes for titles with spaces
  * Task IDs are permanent (even after deletion)
  * Completed tasks remain in the list
  * Export regularly to backup your tasks

{'='*60}
Author: {__author__}
Version: {__version__}
{'='*60}
"""
    print(help_text)


def main():
    """Main entry point - parse arguments and route to functions.

    Handles command-line argument parsing and routes to appropriate
    function based on the command.

    Returns:
        None
    """
    # No arguments - show help
    if len(sys.argv) < 2:
        show_help()
        return

    command = sys.argv[1].lower()

    # Route to appropriate function based on command
    if command == "add" and len(sys.argv) >= 3:
        # Parse title and optional priority
        args = sys.argv[2:]
        priority = "medium"

        # Check for --priority flag
        if "--priority" in args:
            try:
                idx = args.index("--priority")
                if idx + 1 < len(args):
                    priority = args[idx + 1].lower()
                    if priority not in ("low", "medium", "high"):
                        print(f"Invalid priority: {priority}")
                        print("Valid options: low, medium, high")
                        return
                    # Remove priority flag and value from args
                    args = args[:idx] + args[idx+2:]
                else:
                    print("Error: --priority requires a value (low,
                        medium, high)")
                    return
            except ValueError:
                pass
```

```
391
392         # Join remaining args as title
393         title = " ".join(args)
394         if title:
395             add_task(title, priority)
396         else:
397             print("Error: Please provide a task title.")
398
399     elif command == "list":
400         list_tasks()
401
402     elif command == "done" and len(sys.argv) >= 3:
403         try:
404             task_id = int(sys.argv[2])
405             complete_task(task_id)
406         except ValueError:
407             print("Error: Task ID must be a number.")
408
409     elif command == "delete" and len(sys.argv) >= 3:
410         try:
411             task_id = int(sys.argv[2])
412             delete_task(task_id)
413         except ValueError:
414             print("Error: Task ID must be a number.")
415
416     elif command == "search" and len(sys.argv) >= 3:
417         query = " ".join(sys.argv[2:])
418         search_tasks(query)
419
420     elif command == "export":
421         filename = sys.argv[2] if len(sys.argv) >= 3 else "
               tasks_export.txt"
422         export_tasks(filename)
423
424     elif command == "help":
425         show_help()
426
427     else:
428         print(f"Unknown command: {command}")
429         print("Run 'python taskflow.py help' for usage information.")
430
431
432 if __name__ == "__main__":
433     main()
```

Listing 12.2: Complete storage.py - Data Persistence Module

```
1  """Storage module for TaskFlow - handles JSON data persistence.
2
3  This module provides functions to load and save tasks to a JSON file.
4  Using JSON makes our data human-readable and easy to debug.
5
6  The data structure is:
7  {
8      "tasks": [
9          {
10             "id": 1,
```

```
            "title": "Example task",
            "priority": "medium",
            "completed": false,
            "created": "2026-01-15T10:30:00.123456",
            "completed_at": "2026-01-15T11:00:00.123456"  # optional
        },
        ...
    ],
    "next_id": 2
}

Author: Oussama BOUSSAHLA
Version: 1.1.0
"""

import json
from pathlib import Path

# Path to our data file
TASKS_FILE = Path("tasks.json")


def load_tasks():
    """Load tasks from the JSON file.

    Returns:
        dict: Contains 'tasks' list and 'next_id' counter.
              Returns default structure if file doesn't exist.

    Example:
        >>> data = load_tasks()
        >>> print(data['next_id'])
        1
        >>> print(len(data['tasks']))
        0
    """
    if TASKS_FILE.exists():
        try:
            with open(TASKS_FILE, 'r', encoding='utf-8') as f:
                data = json.load(f)

                # Validate data structure
                if not isinstance(data, dict):
                    print("Warning: Invalid data format. Creating new
                        task list.")
                    return {"tasks": [], "next_id": 1}

                if "tasks" not in data or "next_id" not in data:
                    print("Warning: Missing fields. Creating new task
                        list.")
                    return {"tasks": [], "next_id": 1}

                return data

        except json.JSONDecodeError as e:
            print(f"Warning: Could not parse {TASKS_FILE}: {e}")
            print("Creating new task list.")
            return {"tasks": [], "next_id": 1}
```

```
67
68          except IOError as e:
69              print(f"Warning: Could not read {TASKS_FILE}: {e}")
70              print("Creating new task list.")
71              return {"tasks": [], "next_id": 1}
72
73      # File doesn't exist - return empty structure
74      return {"tasks": [], "next_id": 1}
75
76
77  def save_tasks(data):
78      """Save tasks to the JSON file.
79
80      Args:
81          data (dict): Dictionary with 'tasks' list and 'next_id'
                counter.
82
83      Raises:
84          IOError: If the file cannot be written.
85
86      Example:
87          >>> data = {"tasks": [], "next_id": 1}
88          >>> save_tasks(data)
89      """
90      try:
91          # Validate data structure before saving
92          if not isinstance(data, dict):
93              raise ValueError("Data must be a dictionary")
94
95          if "tasks" not in data or "next_id" not in data:
96              raise ValueError("Data must contain 'tasks' and 'next_id'
                    fields")
97
98          if not isinstance(data["tasks"], list):
99              raise ValueError("'tasks' must be a list")
100
101         if not isinstance(data["next_id"], int):
102             raise ValueError("'next_id' must be an integer")
103
104         # Write to file with pretty formatting
105         with open(TASKS_FILE, 'w', encoding='utf-8') as f:
106             json.dump(data, f, indent=2, ensure_ascii=False)
107
108     except IOError as e:
109         print(f"Error: Could not write to {TASKS_FILE}: {e}")
110         raise
111
112     except ValueError as e:
113         print(f"Error: Invalid data structure: {e}")
114         raise
115
116
117 def get_task_by_id(task_id):
118     """Get a specific task by its ID.
119
120     Args:
121         task_id (int): The ID of the task to retrieve
122
```

```
123     Returns:
124         dict or None: The task dictionary if found, None otherwise
125     """
126     data = load_tasks()
127     for task in data["tasks"]:
128         if task["id"] == task_id:
129             return task
130     return None
131 
132 
133 def get_statistics():
134     """Get statistics about tasks.
135 
136     Returns:
137         dict: Statistics including total, completed, pending, by
                priority
138     """
139     data = load_tasks()
140     tasks = data["tasks"]
141 
142     stats = {
143         "total": len(tasks),
144         "completed": sum(1 for t in tasks if t["completed"]),
145         "pending": sum(1 for t in tasks if not t["completed"]),
146         "by_priority": {
147             "high": sum(1 for t in tasks if t["priority"] == "high"),
148             "medium": sum(1 for t in tasks if t["priority"] == "
                    medium"),
149             "low": sum(1 for t in tasks if t["priority"] == "low")
150         }
151     }
152 
153     return stats
```

13 Final Exercise

Put everything together with this comprehensive exercise.

13.1 The Challenge

Complete these steps to demonstrate your Git mastery:

1. **Create a feature branch** called `feature/export`
2. **Implement** an export function that saves tasks to a text file:

```python
def export_tasks(filename="tasks_export.txt"):
    """Export all tasks to a text file."""
    data = load_tasks()
    with open(filename, 'w') as f:
        f.write(f"TaskFlow Export - {datetime.now()}\n")
        f.write("=" * 40 + "\n\n")
        for task in data["tasks"]:
            status = "[X]" if task["completed"] else "[ ]"
            f.write(f"{status} #{task['id']} {task['title']}\n")
    print(f"Exported {len(data['tasks'])} tasks to {filename}")
```

3. **Commit** with a proper message: `feat: Add task export functionality`
4. **Switch to main** and add a small change (e.g., update version to 1.1.0)
5. **Commit** that change
6. **Rebase** your feature branch onto main
7. **Merge** the feature into main (should be fast-forward now)
8. **Tag** this as `v1.1.0` with an annotated message
9. **Push** everything to your remote

13.2 Verification

After completing the exercise, verify your work:

```
# Check for clean linear history
git log --oneline --graph -10

# Verify tag exists
git tag -l "v1.1.0"
git show v1.1.0

# Confirm remote is updated
git log origin/main --oneline -5

# Test the new feature
python taskflow.py add "Test task"
python taskflow.py export
cat tasks_export.txt
```

13.3 Self-Assessment Questions

Test your understanding:

1. What's the difference between `git reset -soft` and `git reset -hard`?
2. Why should you never rebase commits that have been pushed to a shared branch?
3. What does HEAD point to in normal mode vs detached mode?
4. When would you use `git revert` instead of `git reset`?
5. What is Git's default merge strategy as of Git 2.33?
6. How do you recover a commit that you accidentally "lost"?
7. What's the difference between a GitHub Pull Request and a GitLab Merge Request?

14 Conclusion

14.1 What You've Mastered

Congratulations! By building TaskFlow, you've learned:

- **The Three Trees**: Working directory, staging area, and repository
- **Branching**: Creating, switching, and managing lightweight pointers
- **Merging**: Fast-forward and three-way merges with the ORT strategy
- **Conflicts**: Creating and resolving merge conflicts confidently
- **Rebasing**: When and how to rewrite history safely
- **Undoing**: Reset, revert, and restore for different situations
- **Tags**: Marking releases with semantic versioning
- **GitHub**: Pull requests, issues, and Actions for CI/CD
- **GitLab**: Merge requests and GitLab CI/CD pipelines

14.2 The Mental Model

Five Truths About Git

1. **Commits** are nodes in a directed acyclic graph (DAG) – they point to their parents but never forward
2. **Branches** are just pointers to commits – cheap, fast, and movable
3. **HEAD** tells Git where you are – it usually points to a branch
4. **Tags** are immutable pointers – they mark important points forever
5. **History is immutable** – you can only add to the graph, never truly delete (for 90 days, at least)

14.3 Next Steps

Continue your Git journey with these advanced topics:

- `git stash`: Temporarily save work-in-progress
- `git bisect`: Binary search to find bug-introducing commits
- `git cherry-pick`: Apply specific commits from other branches
- `git submodules`: Include other repositories as dependencies
- `git hooks`: Automate actions on commit, push, etc.
- `git worktrees`: Work on multiple branches simultaneously

Keep Practicing

The best way to solidify your Git knowledge:

- **Contribute to open source**: Find a project on GitHub and submit a PR
- **Break things**: Create a test repository and try to break it, then fix it
- **Use Git for everything**: Notes, configurations, dotfiles – version control it all
- **Learn from mistakes**: When something goes wrong, understand *why* before fixing it

Happy Coding!

Made by Oussama BOUSSAHLA

Remember: Git is more forgiving than you think.
The reflog is your friend.